第7章 用例图建模

7.1概述

- 用例图是从现实世界到概念世界的抽象，它是问题求解过程中，到达计算机世界的第一步。
- 它也是软件开发的初期阶段建立的模型，其来源是用户的需求。
- 它不涉及具体的计算机技术，主要目的是在软件设计人员与用户之间进行沟通，同时也为软件设计人员之间的沟通提供了一种介质。

本章主要讲述用例图模型

7.1.1 交互建模

用**交互建模**描述系统内部的交互，即描述了系统内部哪些对象之间需要通信以及通信的次序。交互模型由用例图、顺序图、状态图以及活动图组成。它们从不同的层次上来描述对象之间的交互过程。

用例图描述外部参与者如何与系统的用例进行交互，它是最上层的交互；顺序图提供更多的细节，并显示一组对象之间随着时间变化所完成的交互，顺序图更适于显示用户观测到的行为序列，它是中间层的交互；状态图提供了更细节的交互，它显示单个对象的行为，即一个对象在事件的驱动下，在对象整个生命周期内的变化，它是下层的交互；活动图从另外一个角度来描述交互，它侧重于控制流程的描述，为用例图、顺序图以及状态图提供一种描述补充。活动图既可以显示某次计算中步骤之间的控制关系，也可以表示表述系统的数据流或者控制流

7.1.2 用例图

用例图站在用户的角度描述用户对系统的需求，主要用来描述角色以及角色与用例之间的连接关系。从动态的侧面来描述系统提供给用户可使用的功能。说明是谁要使用系统，以及他们使用这个系统干什么。在系统开发的初期阶段把握用户需求时使用，便于用户和系统开发者之间进行意向沟通。

当用例图出现在外部用户前的时候，它捕获到系统、子系统或类的行为。它将系统功能划分成对参与者(即系统的目标用户)有用的需求。而交互功能部分被称作用例。用例使用系统与一个或多个参与者之间的一系列消息来描述系统中的功能性交互。参与者是一类角色，它可以是人,也可以是外部计算机系统和外部进程。

7.1.2 用例图

用例图处于分析设计阶段的前奏，以用例图为基础，来做成其他UML模型，例如顺序图，通信图，类图等。

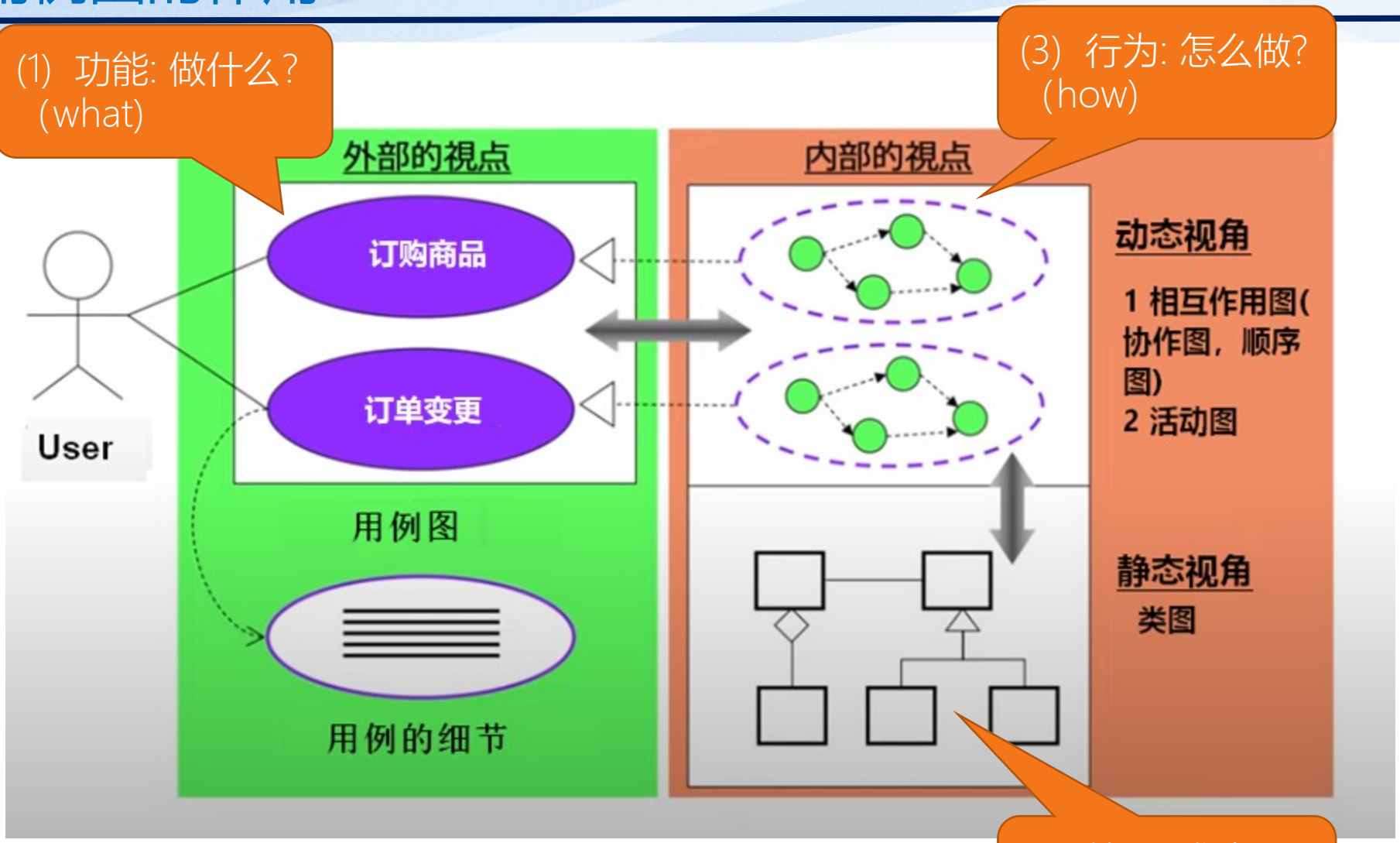
一个用例图包含了多种模型元素，如系统、参与者和用例，并且显示这些元素之间的各种关系，如泛化、关联和依赖。它展示了一个外部用户能够观察到的系统功能模型图，帮助开发团队以一种可视化的方式理解系统的功能需求。故，用例图由以下元素组成：

- 参与者（actor）
- 用例（usecase）
- 关联
- 系统边界

用例图的目的

- 在系统开发的初期阶段，基于以下目的做成用例图：
  - 明确开发系统的主要功能。
  - 明确开发对象范围。
  - 明确开发对象和外界的关系。

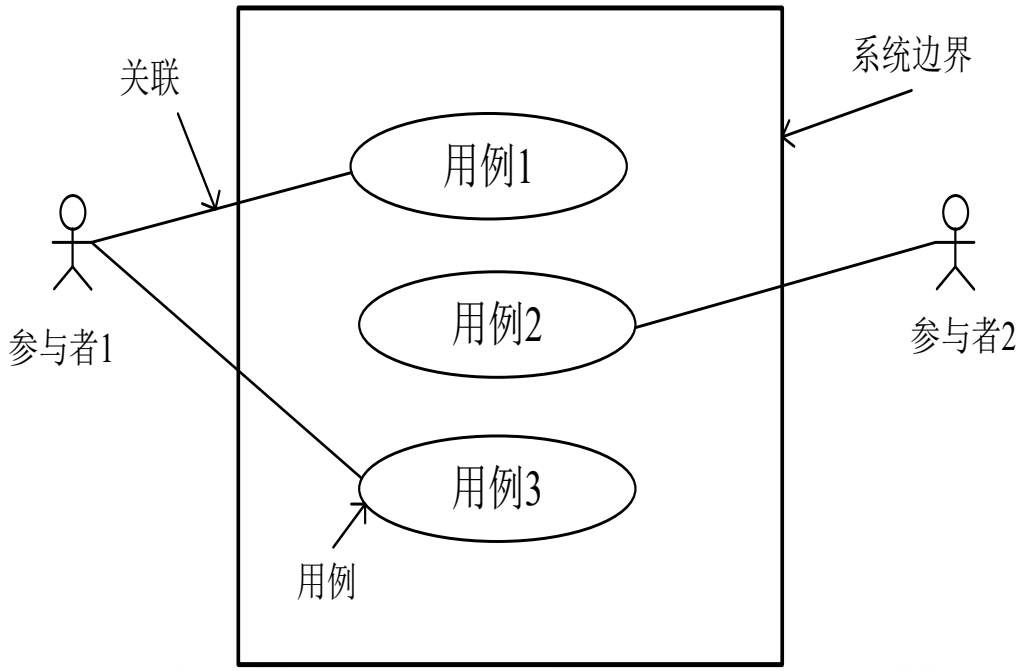
用例图的作用



- 用例图显示从外部要素看,系统提供什么样的功能或者服务

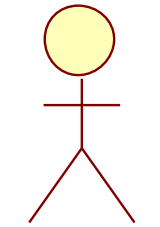
7.2 UML用例图描述

使用UML建模用例图模型时，需要表明参与者、用例、系统边界以及参与者与用例的关联，如图7.1所示。图中包含3个用例，即用例1、用例2和用例3；有2个参与者，即参与者1和参与者2；参与者1与用例1和用例3有交互，但与这2与用例2有交互，整个系统的边界将参与者与用例进行隔开，表示参与者不是系统的功能。

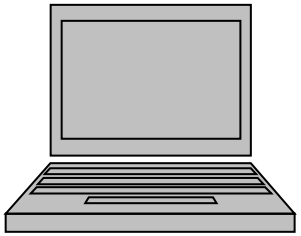


7.2.1参与者

- 参与者是与对象系统进行信息交换的人，物，或者其他系统。
- 参与者存在于对象系统之外，既可主动与对象系统进行信息交换，也可以被动接受来自于对象系统的信息。。
- 参与者的表示方法





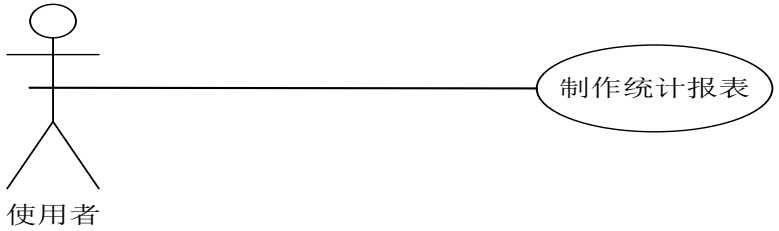


客户管理系统

- 可以成为参与者的有：
  - 系统使用者
  - 外部硬件
  - 现有的系统

参与者的例子

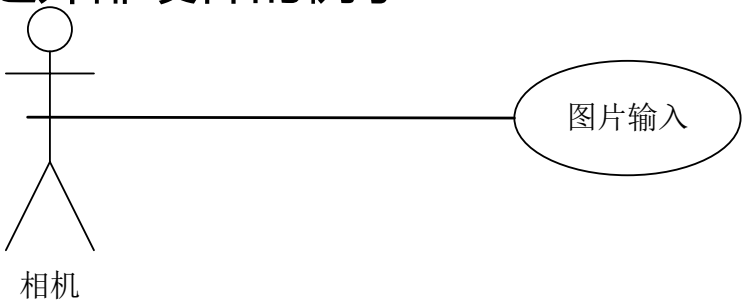
- 参与者是使用者的例子

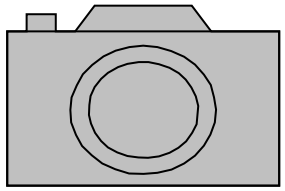




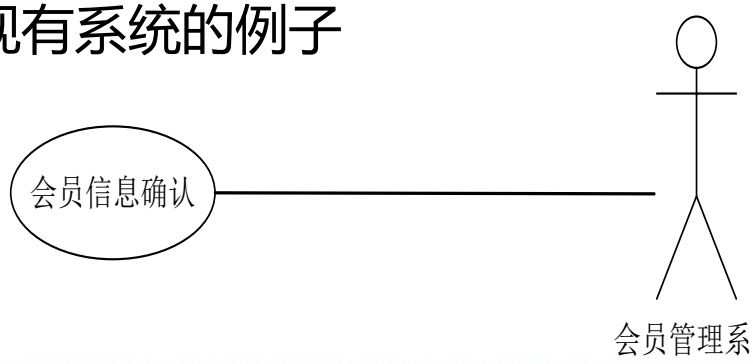
注：参与者是一个角色，并不是某个个别人。

- 参与者是外部硬件的例子





- 参与者是现有系统的例子





参与者的例子

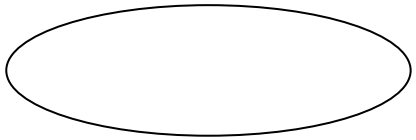
- 用例图主要强调功能和参与者之间的交互，而不是过程的时间顺序。
- 但是用例图可以有“时间参与者"
  - 比如定时器（timer）或者计划任务（scheduled task），来代表时间触发的事件，这些时间触发者可以在用例中执行某些操作，从而引发用例的执行。
  - 例如，在一个报告生成系统的用例图中，可以有一个名为“定时报告生成器”的参与者，代表每天的某个特定时间自动触发报告生成的用例。
  - 时间参与者一般应理解为处于系统边界之外。

用例

- 用例是站在使用者的立场上看到的系统所提供的功能。
  - 是系统中的功能。
  - 一个用例表示一个功能，集中所有的用例，可完整描述该如何使用该系统。
  - 可以通过关联线与参与者连接，一个用例至少要与一个参与者相关联。
  - 给用例起名要站在使用者的立场上考虑
  - 可以用系统边界把用例框起来以区分系统内外。

- 用例的表示方法





用例名称

用例例子

参与者与系统的不同的交互都可以被量化为用例。
自动售货机例子。

- **购买一瓶饮料。**在客户选择并支付以后，自动售货机弹出饮料。
- **进行定期维修。**维修技师对自动售货机进行定期服务，以让它保持良好的工作状态。
- **维修。**维修技师要对自动售货机做一些非预期的工作，排除运行过程中出现的故障。
- **装入物品。**存货管理员给自动售货机添加物品，补给饮料供应。

- **用例**是代表系统中各个项目相关人员之间就系统的行为所达成的契约。

例：在一个银行业务系统中，可能会有以下一些**用例**：

- 浏览帐户余额
- 列出交易内容
- 划拨资金
- 支付帐款
- 登录
- 退出系统
- 编辑配置文件
- 买进证券
- 卖出证券

use case说明：

◆Use case从使用系统的角度描述系统中的信息，即站在系统外部察看系统功能，并不考虑系统内部对该功能的具体实现方式。

◆使用use case可以促进与用户沟通，理解正确的需求，同时也可以用来划分系统与外部实体的界限，是OO系统设计的起点，是类、对象、操作的来源。

◆用例的一些特点：

（1）用例描述了用户提出的一些可见的需求；

（2）用例可大可小；

（3）用例对应一个具体的用户目标。

◆理论上可以把一个软件系统的所有Use Case画出来，但实际运用时只需把重要的、交互过程复杂的那些画出来。

◆需求有两种基本形式：功能性和非功能性的。不是所有的需求都要用use case表示出来。

◆问题：一个系统的需求包括哪些内容？

■可以参照需求大纲

- Use Case技术很容易使用，但也很容易误用。
- 正确使用Use Case 技术来做好Domain Modeling，以确保定义出正确的需求(right requirements)，然后开发出正确的系统(right system)，是保证OO软件开发成功的基础。

争论：

- 本质上，Use Case分析是一种**功能分解(functional decomposition)**的技术，没有以对象观点为中心，因而有人认为Use Case分析只是OOA/OOD的先导性工作，并非OOA/OOD过程的一部分；但也有人视其为OOA/OOD的一环。
- 不管怎样，Use Case是UML的一部分，Use Case是OO开发过程中的第一步。

用例特点

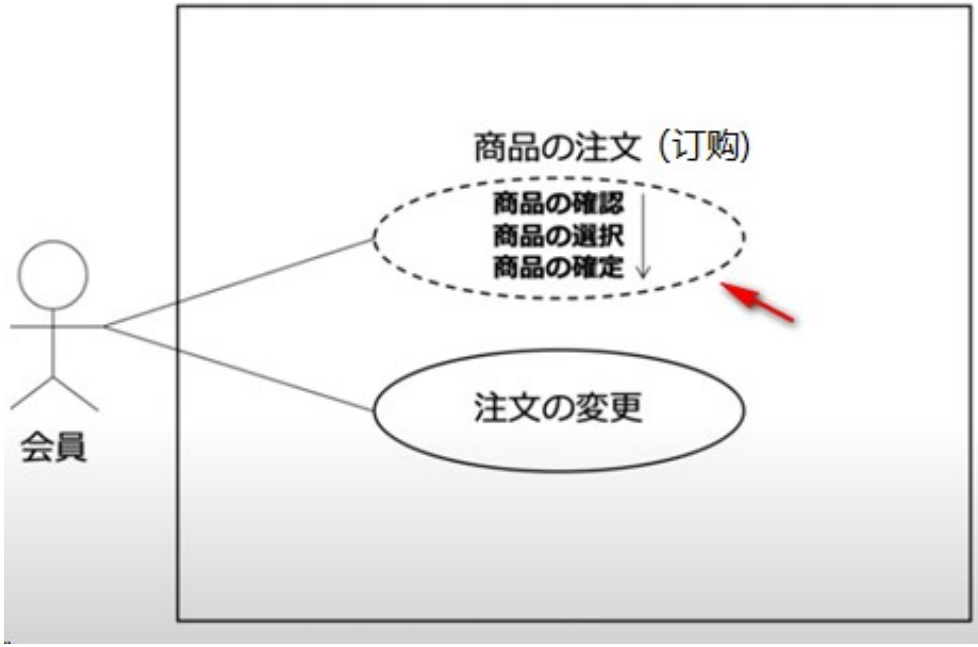
1 每个用例会涉及到一个或多个参与者以及系统本身。

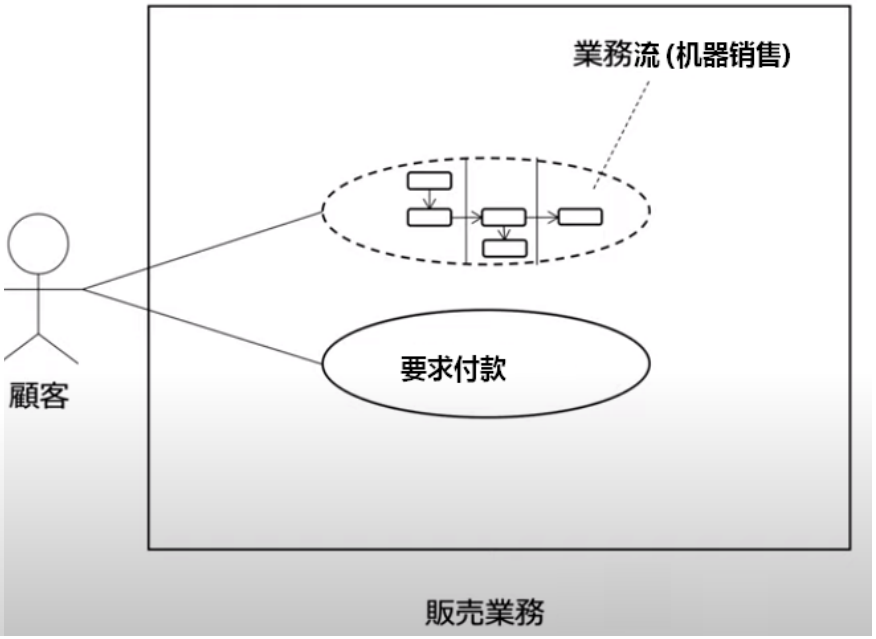
2 用例涉及系统与其参与者之间的消息序列。

3 错误条件也是用例的一部分。

◆用例的要点

- ■首先用例以向参与者提供的有价值的服务单位。
- ■为了提供有价值的服务，用例内部有自己的流程。

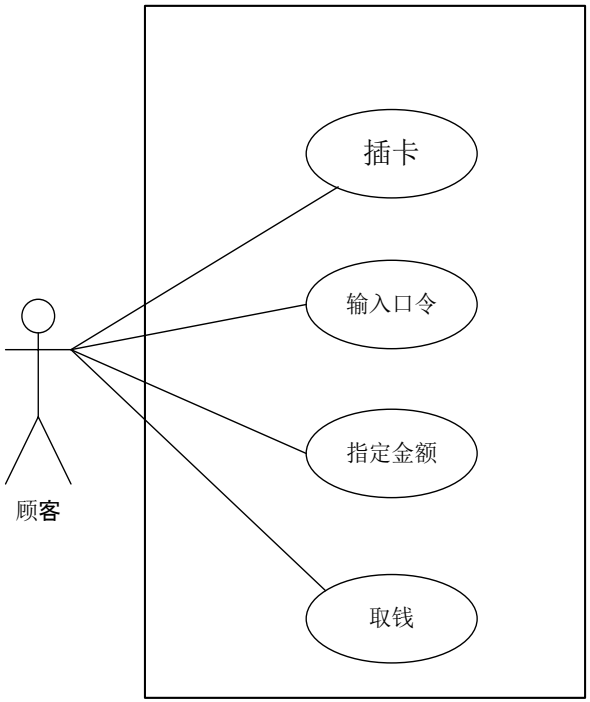




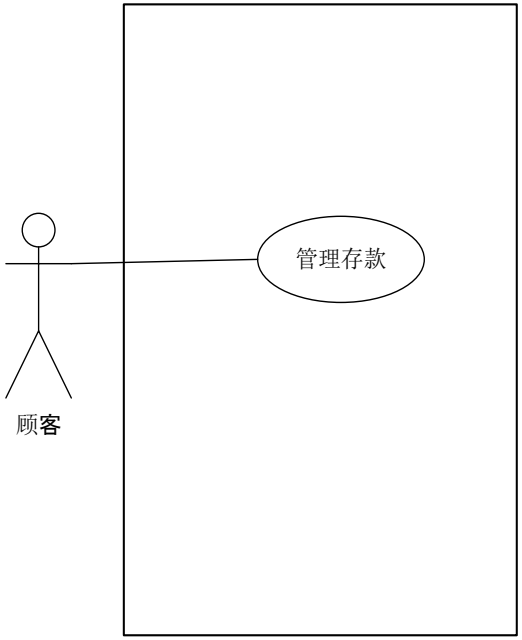
7.2.3用例大小的设定

- 对系统中的用例进行分割的时候，要站在用户的立场上按如下原则进行分割：
  - 一个用例结束，代表用户使用该系统的一个目的达到。
  - 一个用例在执行过程中不宜发生中断。

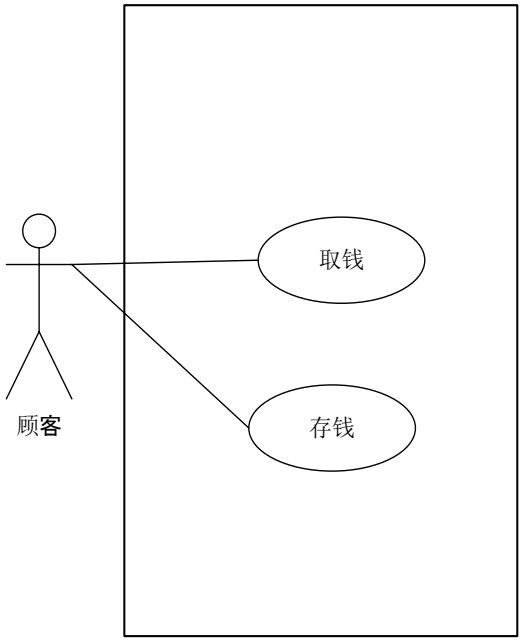
使用ATM机系统的例子：



过小



过大



合适

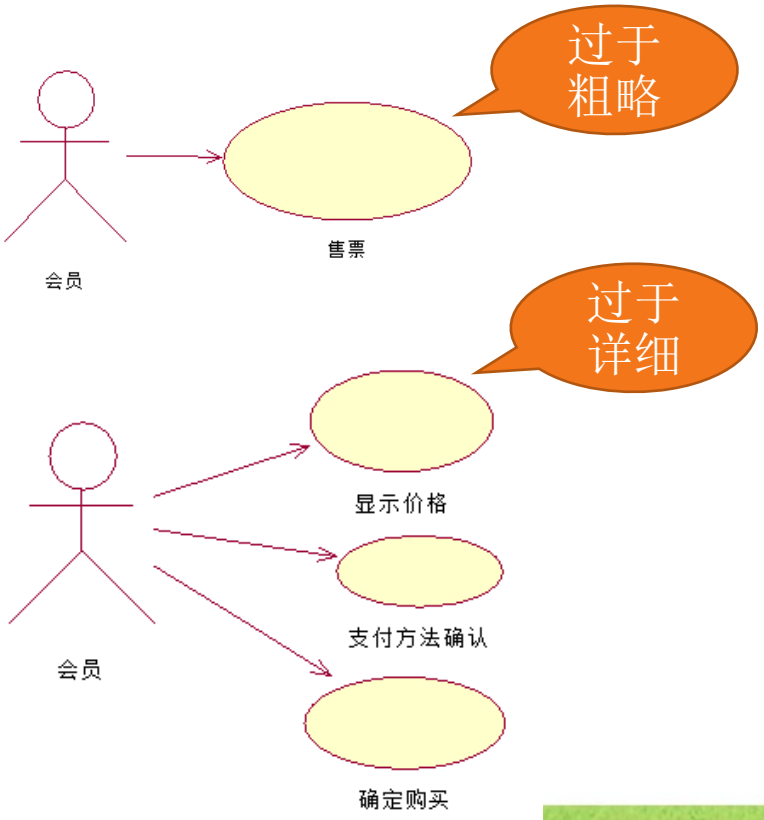
7.2.3用例大小的设定

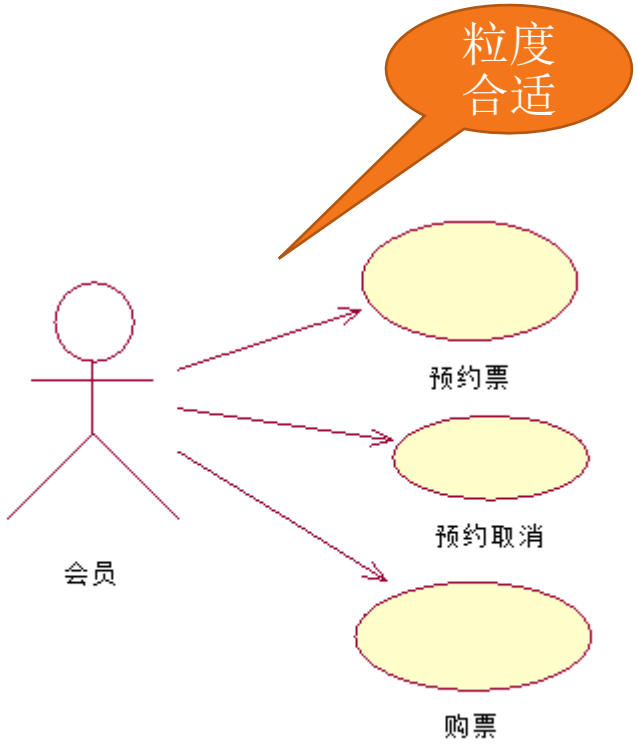
◆以参与者感知到的系统功能的大小为原则。

■一个用例的功能完成后，达到参与者的一个目的。

■一个用例的中间不发生中断。

■以会员售票系统为例





用例模型准则

1 首先确保系统边界。

2 确保关注参与者。每个参与者都应该有单一的、一致的目的。例如，计算机的拥有者会安装软件、配置数据库和发送邮件。由于这些功能有很大的不同，因此可以分为：系统管理员，数据库管理员，计算机用户3个参与者。

3 每个用例必须给用户提供价值。用例应该表示成给用户提供价值的完整事务，不应该被定义的过于狭窄。

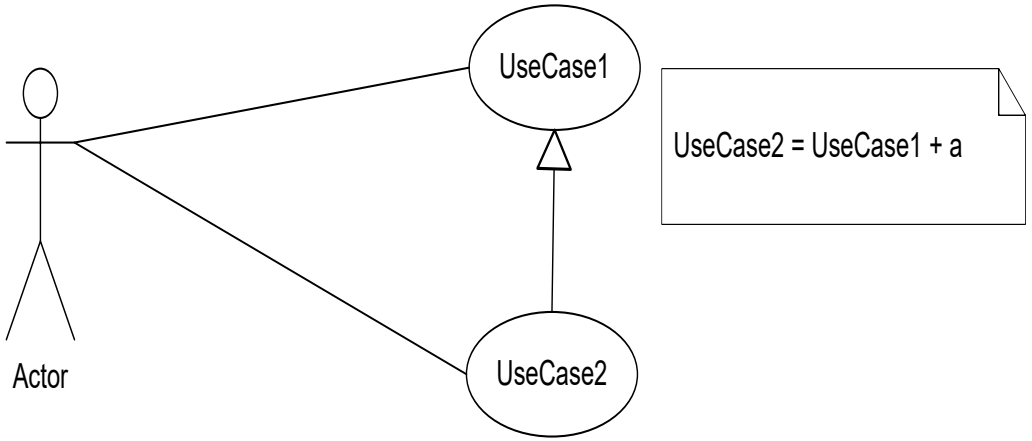
例如，电话系统中，拨电话就不是一个好用例。它仅是打电话用例的一部分。

4 用例是非形式化的。它不是一种形式化的机制，而是从以用户为中心的角度来识别和组织系统功能。

7.2.4用例之间的关系

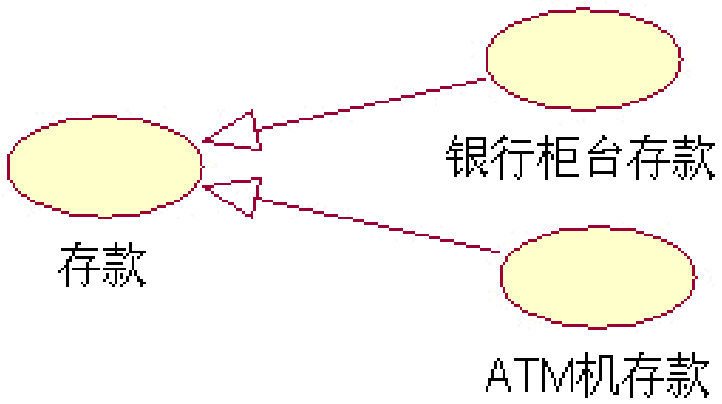
- 泛化:与类之间的泛化类似，泛化是用例的一个特定变体，即子用例特化父用例。
- 一个用例也可以被特别列举为一个或多个子用例，这被称做用例泛化。
- 当父用例能够被使用时，任何子用例也可以被使用。

- 泛化可能是步骤的增加，一个用例的流程默认情况下总是被另外一个用例所使用。



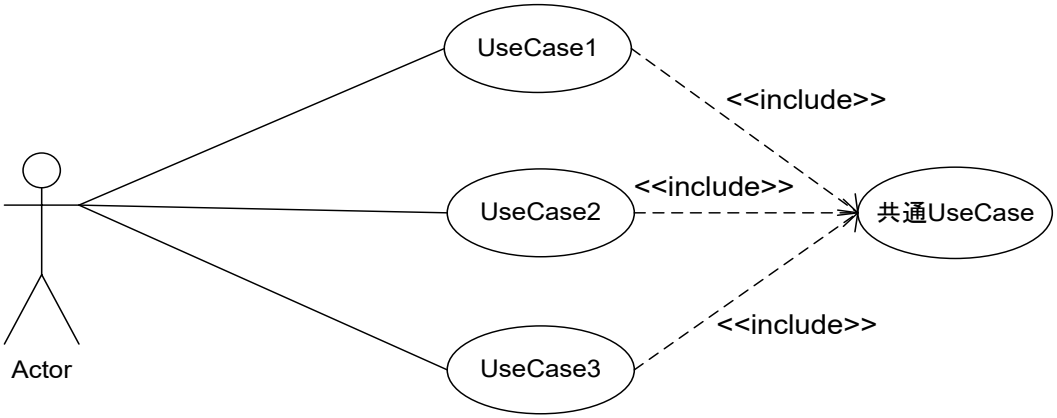
7.2.4用例之间的关系

◆泛化的示例：银行存款有两种方式，一种是银行柜台存款，一种是ATM机存款。在这里，银行柜台存款和ATM机存款都是存款的一种特殊方式，因此“存款”为父用例，“银行柜台存款”和“ATM机存款”为子用例。



7.2.4用例之间的关系

- 包含（include）
  - 多个用例的共通部分进行分离，用包含（include）关系进行连接。



7.2.4用例之间的关系

◆包含关系代表着基础用例会用到被包含用例，具体的讲就是将被包含用例的事件流插入到基础用例的事件流中。需要注意的是，包含关系是UML1.3中的表述，在UML1.1中，同等语义的关系被表述为使用(uses)。



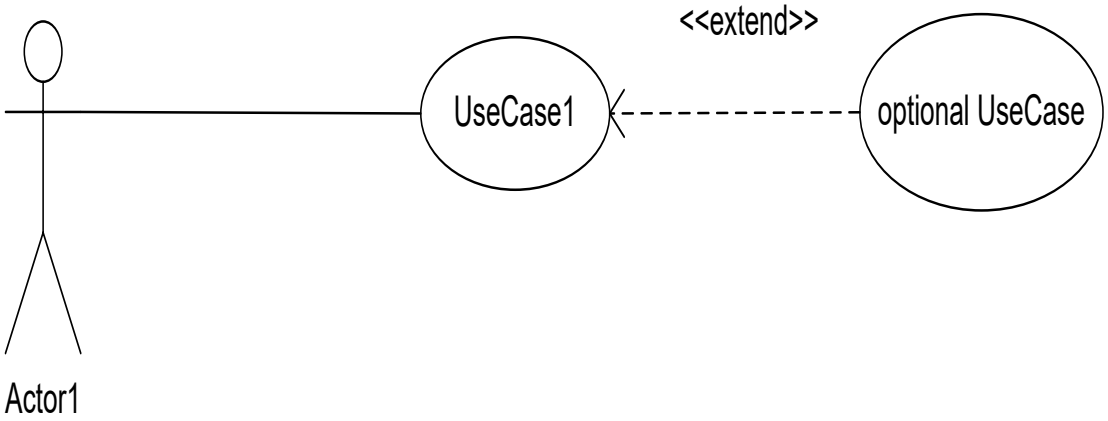
7.2.4用例之间的关系

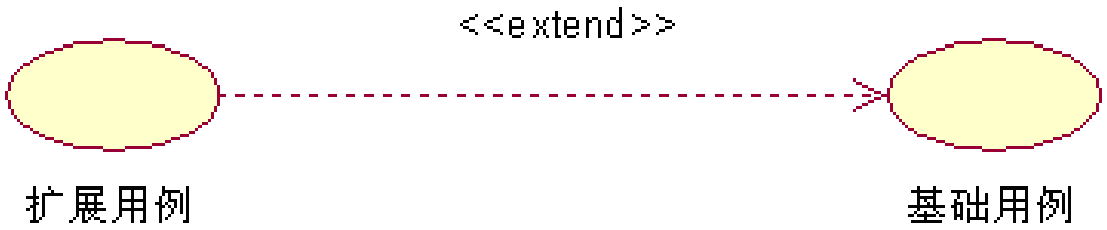
- 在处理包含关系时，具体的做法就是把几个用例的公共部分单独的抽象出来成为一个新的用例。主要有两种情况需要用到包含关系：
- 第一，多个用例用到同一段的行为，则可以把这段共同的行为单独抽象成为一个用例，然后让其他用例来包含这一用例。
- 第二，某一个用例的功能过多、事件流过于复杂时，我们也可以把某一段事件流抽象成为一个被包含的用例，以达到简化描述的目的。

7.2.4用例之间的关系

◆扩张（extend）

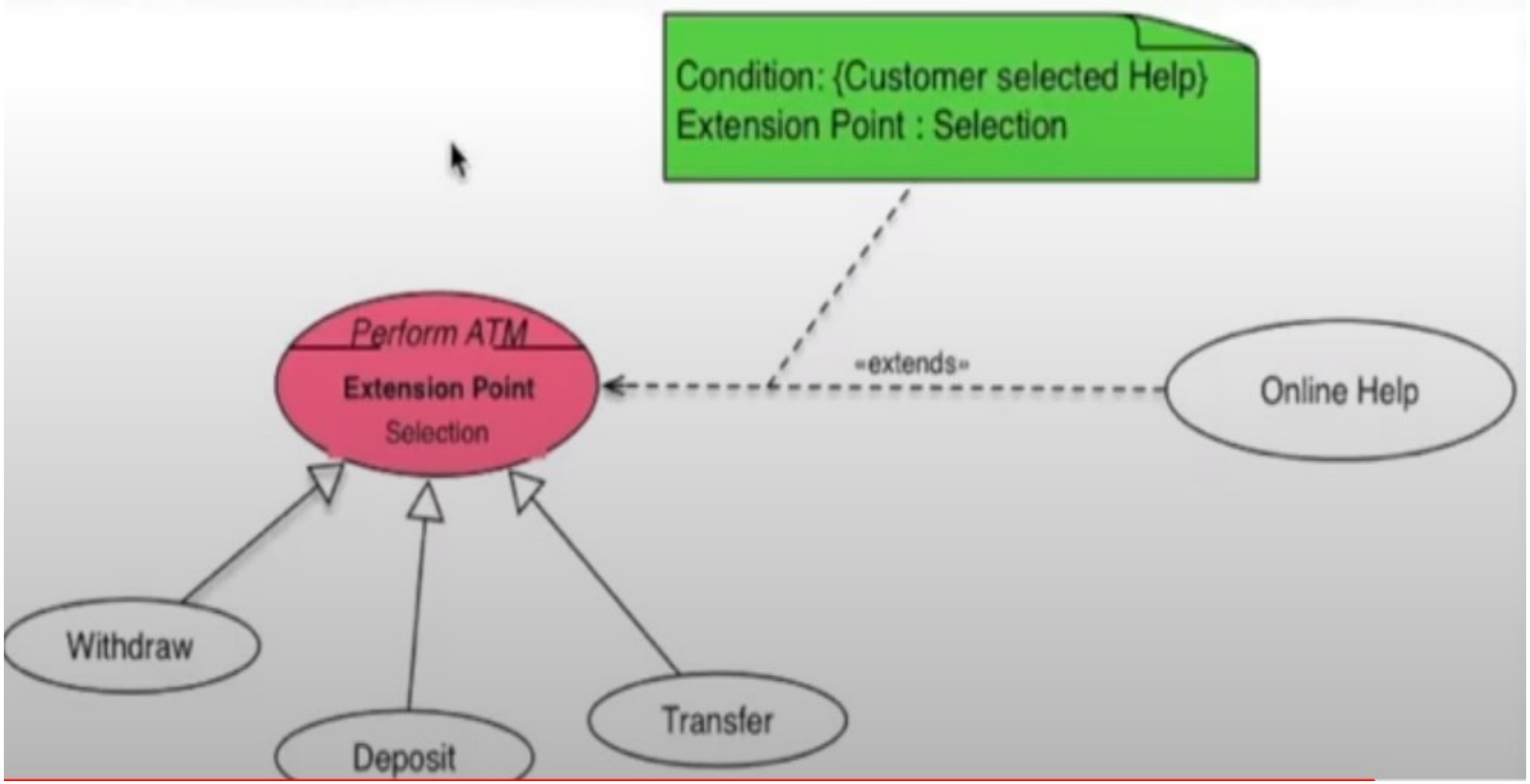
- 有条件有选择地被执行的用例。（注意箭头方向）



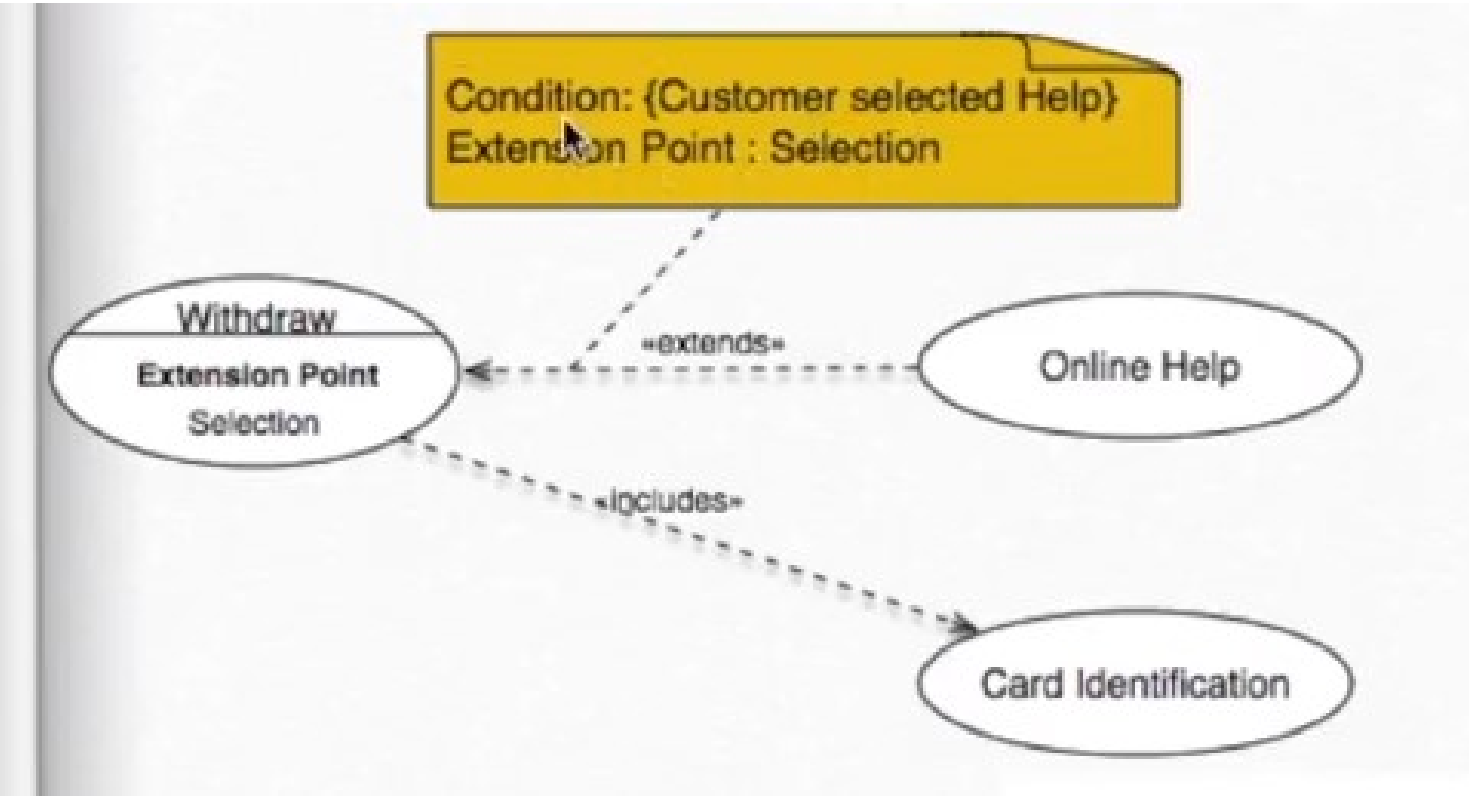


7.2.4用例之间的关系

◆需要时可以标注扩展点, 并在注释中说明扩展条件。如下图所示



◆需要时可以标注扩展点，并在注释中说明扩展条件。如下图所示



用例之间关系举例

◆UML 2.5.1 Specification P646

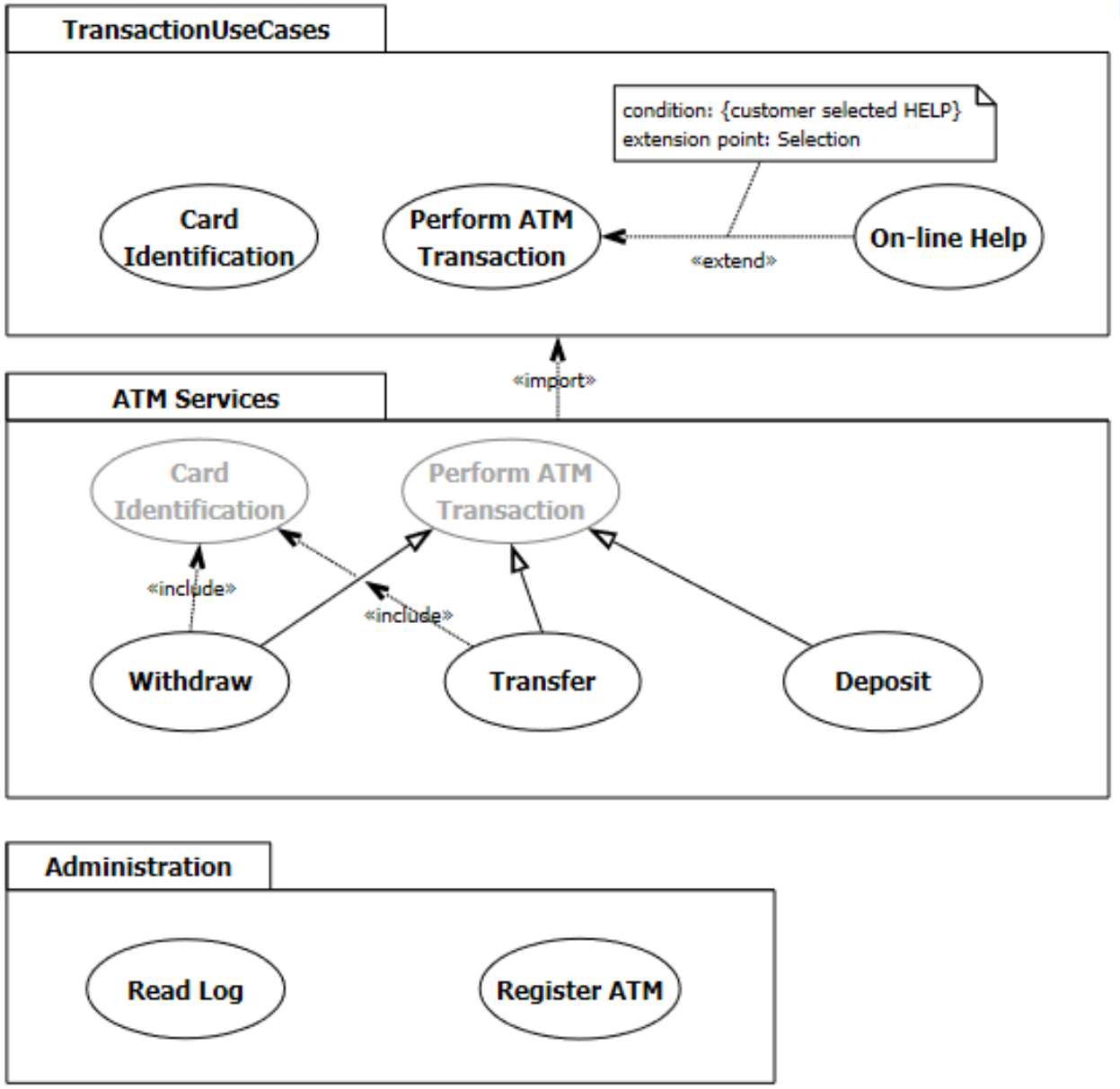


Figure 18.11 Example UseCases owned by Packages

7.2.4用例之间的关系

- ◆包含关系和扩展关系
  - ■被扩展用例是有条件执行；被包含用例是一定执行
  - ■基础用例为扩展用例提供1到多个插入点；而包含关系中插入点只有一个
  - ■扩展关系中，基础用例本身就是完整的；而包含关系中，基础用例离开被包含用例就不完整

7.2.4用例之间的关系

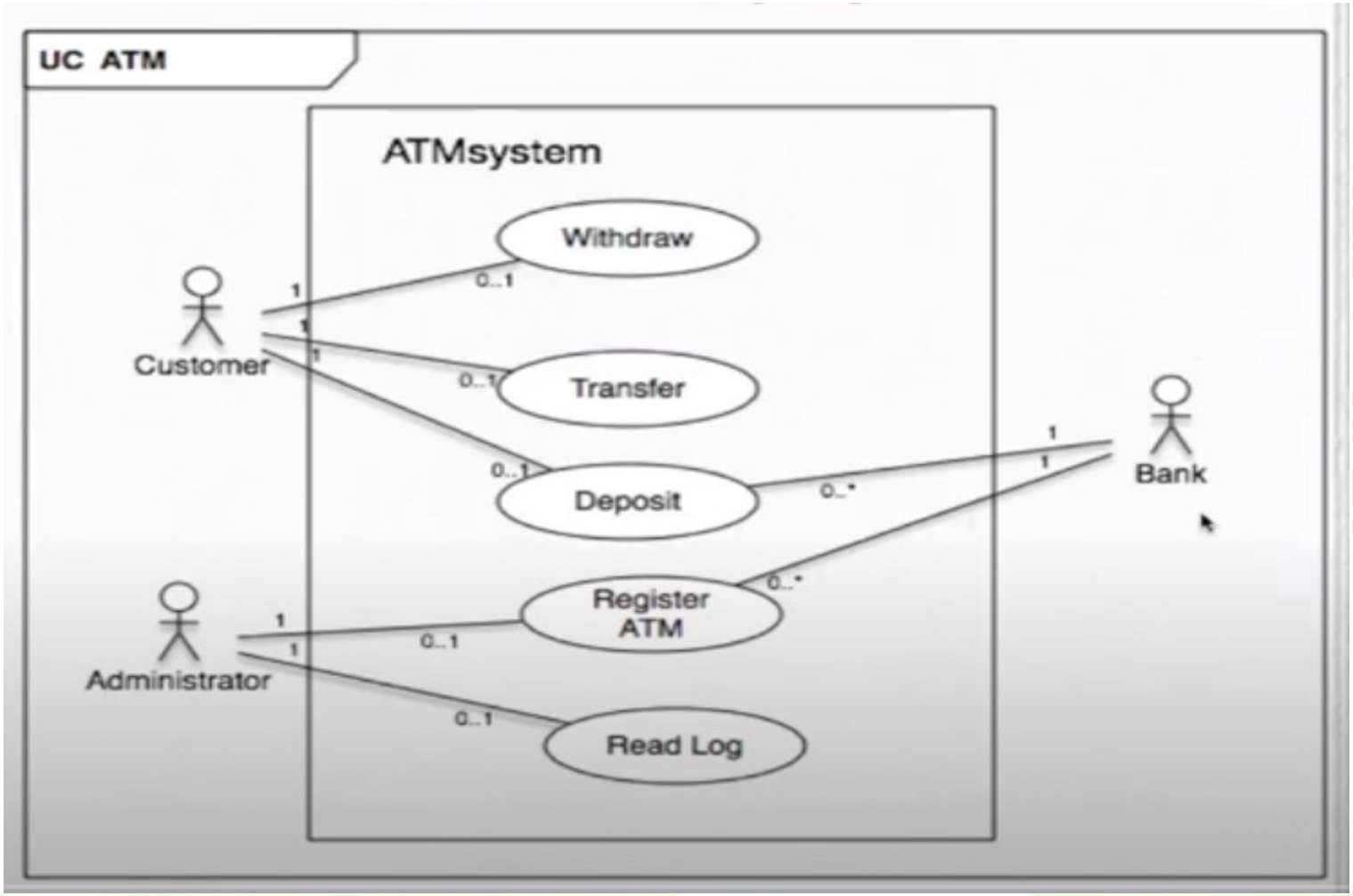
◆用例之间的依赖关系

特性	include	extend
作用	增强基用例的行为	增强基用例的行为
执行过程	包含用例一定会执行	扩展用例可能被执行
对基用例的要求	在没有包含用例的情况下，基用例可以是也可以不是良构的	在没有扩展用例的情况下，基用例一定是良构的
表示法	箭头指向包含用例	箭头指向基用例
基用例对增强行为的可见性	基用例可以看到包含用例，并决定包含用例的执行	基用例对插入不知情
基用例每执行一次，增强行为的执行次数	只执行一次	取决于条件（0到多次）

7.2.5参与者的关系

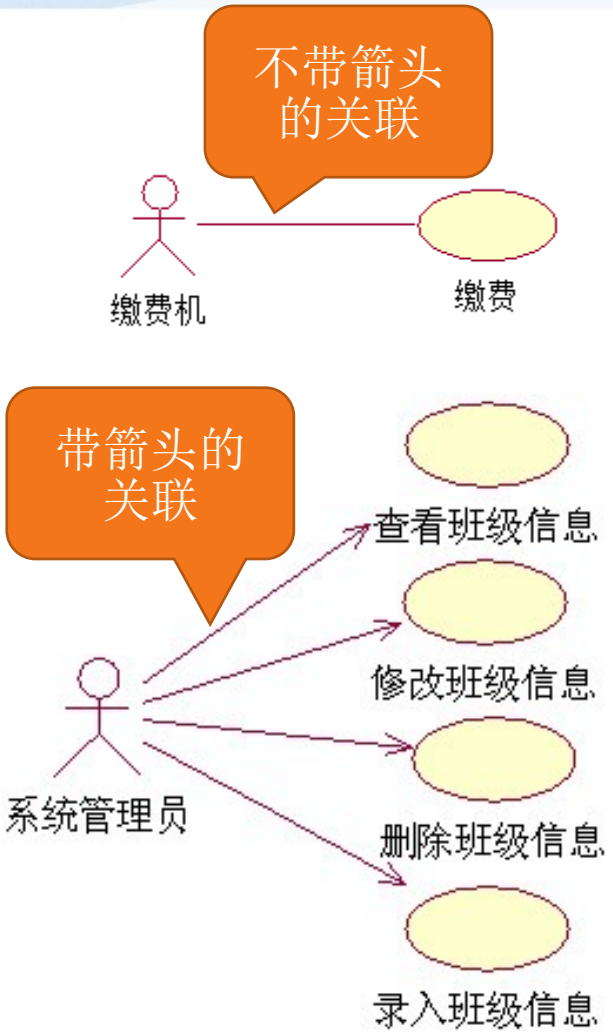
参与者和用例之间的关系关系又叫做“通信关联”

通信关联可以标注多重度。



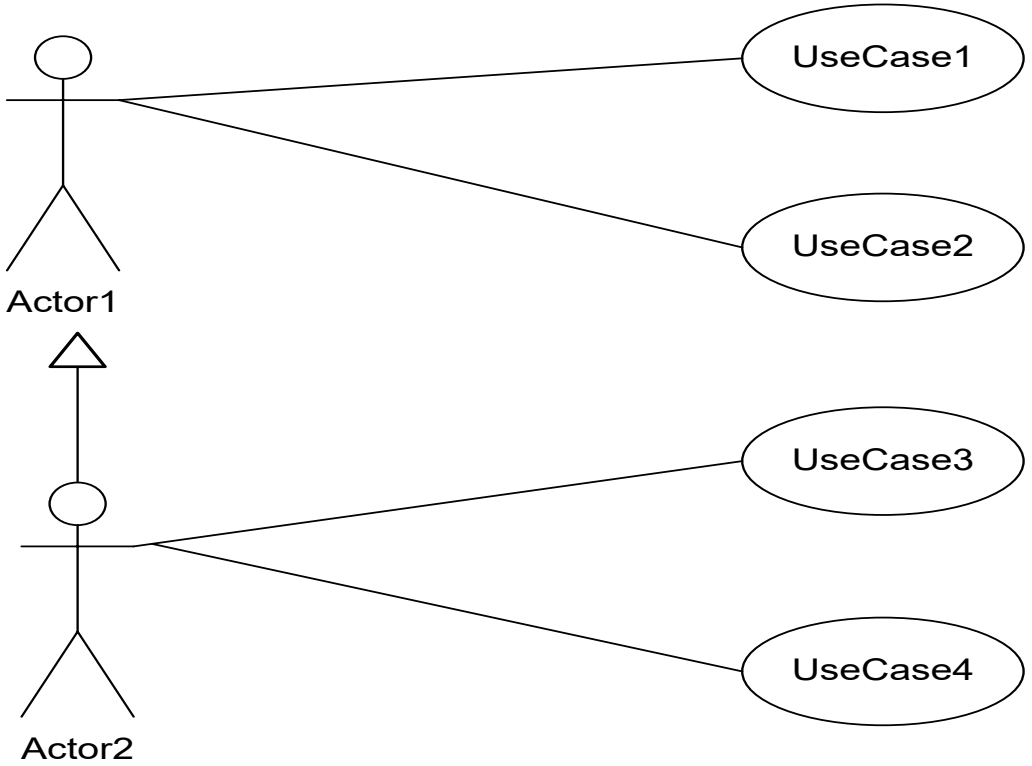
7.2.5参与者的关系

- 通信关联关系是用一条直线表示的，通常不带箭头。这是因为关联本身代表了双向的交互
- 在实际使用过程中，有些画图者可能会在关联上加箭头来表示交互的方向性，尤其是当他们希望表明信息流或者控制流的方向时。



7.2.5参与者的关系

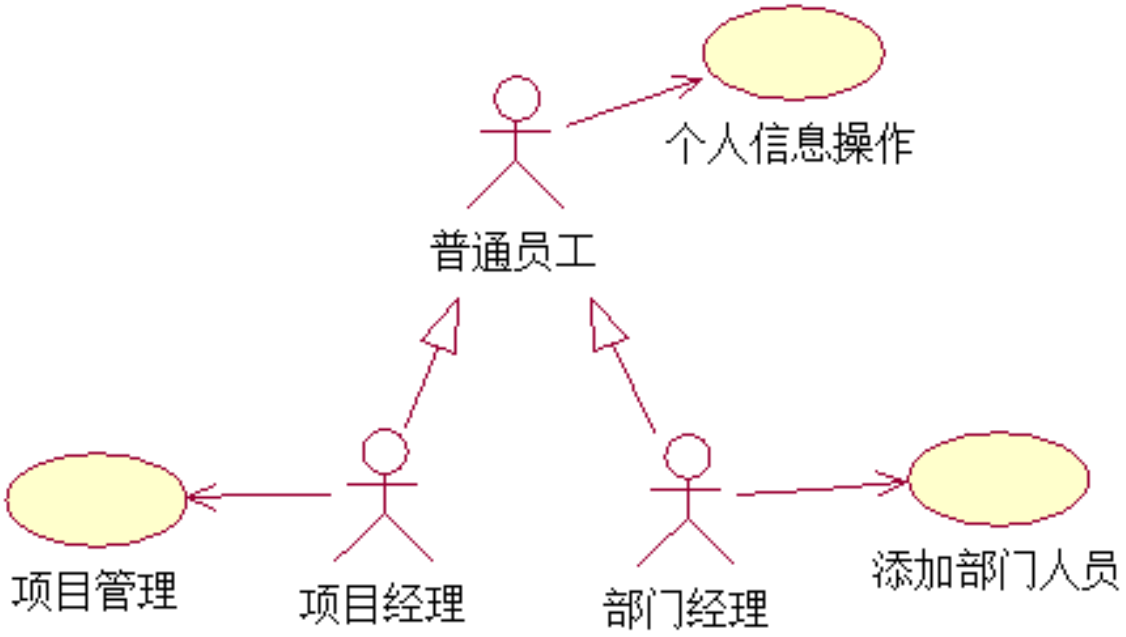
- 参与者之间的泛化关系
  - 某一个参与者相关联的所有用例，都和另外一个参与者相关联。
  - 可减少关联，简化用例图



7.2.5参与者的关系

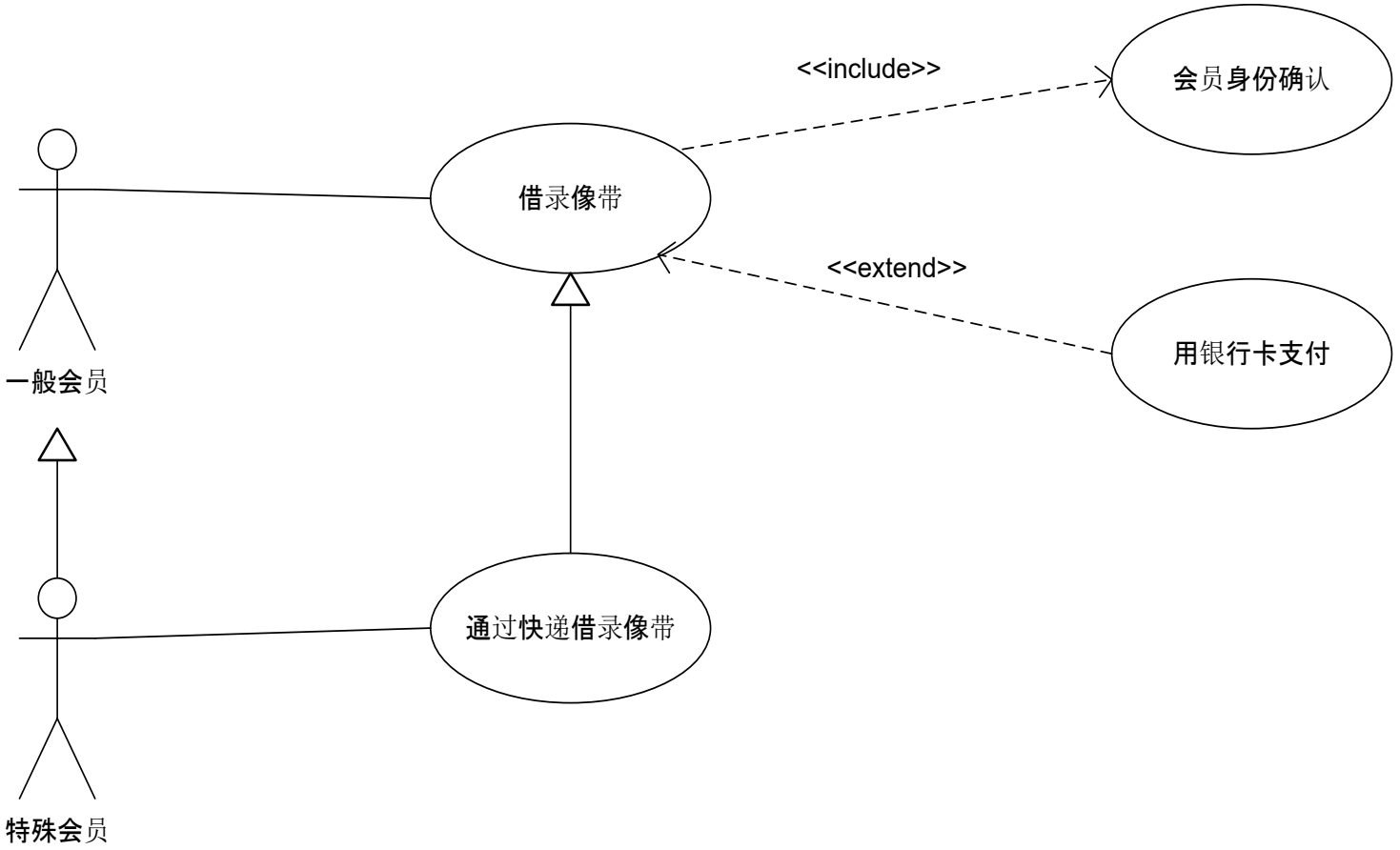
◆ 参与者之间的泛化关系

- 泛化关系的含义是把某些参与者的共同行为提取出来表示成通用行为，并描述成超类。泛化关系表示的是参与者之间的一般/特殊关系



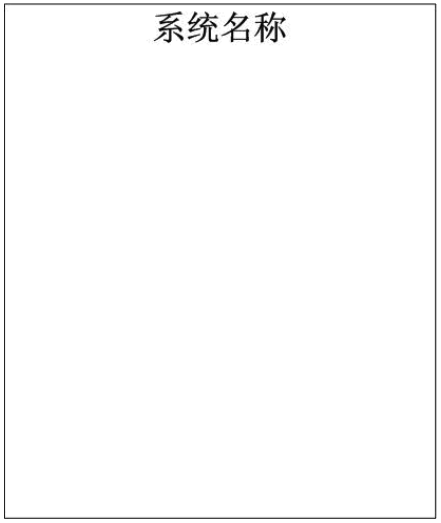
关系的例子

◆ 以录像带租赁业务的例子，来说明参与者之间，用例之间的各种关系。



系统边界

- 在项目开发过程中，边界是一个非常重要的概念。这里说的系统边界是指系统与系统之间的界限。通常我们所说的系统可以认为是由一系列的相互作用的元素形成的具有特定功能的有机整体。
- 系统同时又是相对的，一个系统本身又可以是另一个更大系统的组成部分，因此，系统与系统之间需要使用系统边界进行区分开来。我们把系统边界以外的同系统相关联的其他部分，称之为系统环境。



7.3 用例描述

- 用例描述（又翻译为用例规约, Use Case Specification），是对用例图的补充，用例描述和用例图合称为用例模型。（UseCase Model）
  - 可详细描述用例内部的业务流程。
  - 避免仅靠用例图造成的误解。

- UML中没有明确规定用例描述的标准格式，一般情况下应该包括：
  - 概要

    用几行文字描述用例的作用，目的等。
  - 脚本(Use Case Scenario, 场景)

    描述一个用例在某种条件下的具体执行流程。
  - 事件流

    描述用例在各种条件下的具体流程。

-特殊需求: 特殊需求指的是一个用例的非功能性需求和设计约束。特殊需求通常是非功能性需求，包括可靠性、性能、可用性和可扩展性等。例如法律或法规方面的需求、应用程序标准和所构建系统的质量属性等。

脚本和事件流

- 脚本用一段文字来描述用例执行时的具体流程。分为
  - 基本脚本
    正常运行能达到用例目的的脚本。
  - 例外脚本
    出现异常情况不能达到用例目的的脚本。

- 事件流描述用例在各种条件下的具体执行流程，一般包括：
  - 前提条件
    为使用例开始而必须满足的条件。。
  - 事后条件
    用例结束后对系统造成的影响。
  - 基本流
    正常处理时的基本流程。
  - 代替流
    出现概率低，但用例也还算是正常结束的流程。
  - 例外流
    发生错误时的流程。

用例描述例子

用例名称	提交订单
用例编号	UC002
参与者	会员
用例描述	该用例描述一个系统会员提交一份订单的行为
触发器	当订单被提交时，用例触发。
前置条件	提交订单的一方需要完成登录操作
后置条件	如果订单中的商品有库存，则发货；否则提示用户当前缺货
基本事件流	1 参与者将订单信息提交至系统。 2 系统验证用户信息及订单信息合法后作出响应。 3 对于订单中的每种产品，系统根据订单中的数量检查产品库存数量。 4 系统统计订单中产品的总价格。 5 系统从会员的系统账户余额中扣除相应金额。 6 系统生成并保存订单信息并将订单发送至分销中心。 7 系统生成订单确认页面并发送给会员。
扩展事件流	A-2 如果订单信息非法，系统通知会员并提示重新提交订单。 A-3 如果订单中产品数量超过产品库存量，则提示会员库存不足，暂无法购买，取消订单同时终止用例。 A-5 如果会员账户余额不足，系统给出相应提示，取消订单并终止用例。
结论	当会员收到系统发送的订单确认页面或其他异常信息时，用例结束。
数据需求	D-1 订单信息包括订单号、参与者的会员账户名、商品种类数量、商品种类名称以及每种商品的数量。
业务规则	B-1 只有当订单中商品信息确认无误后才能要求会员进行支付。

用例描述模板

- 用例模板有各种格式。
- 自20世纪90年代早期以来，使用最为广泛的格式是Alistair Cockburn创建的模版。
- 他是用例建模方法和畅销书的《Writing Effective Use Cases》作者。

用例场景

- 同一个用例在实际执行的时候会有很多不同的情况发生，称之为用例场景，也可以说用例场景就是用例的实例。
- 场景是用例在特定条件下执行的某一次具体的流程。因此一定要使用具体的任务、具体的时间、具体的数量。
- 按照时间顺序，简单明了地描述。
- 不过度细化GUI.
- 出现复杂分支时，要对场景进行分割。（分为多个场景）
- 根据包含在用例事件流中的多个分支以及多个代替事件流，可以做出多个场景。
- 场景描述不是必须的，如果有会带来好处
  - 分析时容易发现对象
  - 能够详细地理解系统与用户的交互
  - 方便编写测试用例

场景的例子

◆下面是李世民通过银行柜员机（ATM系统）取款500元的场景如表所示。（这里事件流修改为“场景步骤”更确切. 场景步骤仅描述事件流中的一条具体路径（例如仅基本流，或某个备选流）。是实例化的、线性的步骤序列，不包含分支逻辑。）

场景名称	李世民取款 500 元
参与者	李世民
事件流	

场景步骤

11. 李世民将银行卡插入柜员机
12. 柜员机请求输入密码
13. 李世民输入卡密码，并按确认键
14. 柜员机提示客户：选择服务类型
15. 李世民选择取款服务
16. 柜员机提示客户：选择取款数额
17. 李世民输入：500，并按确认键
18. 柜员机输出 500 元人民币
19. 李世民取回 500 元人民币
20. 柜员机提示客户：继续、退卡

11. 李世民选择服务：退卡

例：用例模板

- **用例名称**：处理订单
- **标识符**：UC1701
- **用例描述**：当一个订单初始化或者被查询的时候这个用例开始。它处理有关订单的初始化定义和授权等问题。当订单业务员完成了同一个顾客的对话的时候，它就结束了。
- **参与者**：订单业务员
- **优先级**：1
- **状态**：通过审查
- **前置条件**：订单业务员登录进系统
- **后置条件**：下订单；库存数目减少

◆基本操作流程：

- ■1. 顾客来订购一个吉他，并且提供信用卡作为支付手段。……
- ■2. …..

◆可选操作流程：

- ■顾客来订购一个吉他，并且使用汇票的方式。……
- ■顾客来订购一个风琴，并且提供信用卡作为支付手段。……
- ■顾客使用信用卡下订单，但那张信用卡是无效的。
- ■顾客来下订单，但他想要的商品没有存货。

◆**被泛化的用例**：无

◆**被包含的用例**：登录 (UC1706)。

◆**被扩展的用例**：无

◆**修改历史记录**：

■张三，定义基本操作流程，2003年5月4日

■张三，定义可选操作流程，2003年5月8日

用例描述的一些常见错误分析

- 在描述用例时易犯的错误包括：
  - 只描述系统的行为，没有描述actor的行为。
  - 只描述actor的行为，没有描述系统的行为。
  - 设定对用户界面的设计的要求。
  - 过于冗长。

◆例1：下面是一个用例描述的片断：

Use Case: Withdraw Cash（提取现金）

参与者：Customer

主事件流：

1. 储户插入ATM卡，并键入密码。
2. 储户按 “Withdrawal” 按钮，并键入取款数目。
3. 储户取走现金、ATM卡并拿走收据。
4. 储户离开。

上述描述中存在的问题：

◆只描述了参与者的动作序列，没有描述系统的行为。

◆改进的描述如下：

Use Case: Withdraw Cash （提取现金）

参与者: Account Holder

主事件流：

1. 通过读卡机，储户插入ATM卡。
2. ATM系统从卡上读取银行ID、帐号、加密密码、并用主银行系统验证银行ID和帐号。
3. 储户键入密码，ATM系统根据上面读出的卡上加密密码，对密码进行验证。
4. 储户按“FASTCASH”按钮，并键入取款数量，取款数量应该是5美元的倍数。
5. ATM系统通知主银行系统，传递储户帐号和取款数量，并接收返回的确认信息和储户帐户余额。
6. ATM系统输出现金，ATM卡和显示帐户余额的收据。
7. ATM系统记录事务到日志文件。

◆例2：下面是一个用例描述的片断：

Use Case: Withdraw Cash

参与者: Customer

主事件流：

1. ATM系统获得ATM卡和密码。
2. 设置事务类型为 "Withdrawal"。
3. ATM系统获取要提取的现金数目。
4. 验证帐户上是否有足够储蓄金额。
5. 输出现金，数据和ATM卡。
6. 系统复位。

上述描述中存在的问题：

◆只描述了ATM系统的行为，而没有描述参与者的行为。这样的描述很难理解，验证和修改。

◆改进的描述同例1。

◆例3：下面是一个用例描述的片断：

Use Case: Buy Something

参与者: Customer

主事件流：

1. 系统显示“ID and Password”屏幕。
2. 顾客键入ID和密码，然后按OK按钮。
3. 系统验证顾客ID和密码，并显示“Personal Information”屏幕
4. 顾客键入姓名、街道地址、城市、州、邮政编码、电话号码，然后按OK按钮。
5. 系统验证用户是否为老顾客。
6. 系统显示可以卖的商品列表。
7. 顾客在准备购买的商品图片上点击，并在图片旁边输入要购买的数量。选购商品完毕后按“Done”按钮。
8. 系统通过库存辅助系统验证要购买的商品是否有足够库存。

...etc.

上述描述中存在的问题：

- 对用户界面的描述过于详细，对于需求文档来说，详细的UI描述对**获取需求**并无帮助。
- 改进的描述可以如下所示：

Use Case: Buy Something

参与者: Customer

主事件流:

1. 顾客使用ID和密码进入系统。
2. 系统验证顾客身份。
3. 顾客提供姓名、地址、电话号码。
4. 系统验证顾客是否为老顾客。
5. 顾客选择要购买的商品和数量。
6. 系统通过库存辅助系统验证要购买的商品是否有足够库存。

... etc.

◆例4：下面是一个用例描述的片断：

Use Case: Buy Something

参与者: user(或Customer)

主事件流：

1. 顾客使用ID和密码进入系统。
2. 系统验证顾客身份。
3. 顾客提供姓名。
4. 顾客提供地址。
5. 顾客提供电话号码。
6. 顾客选取商品。
7. 顾客确定商品的数量。
8. 系统验证顾客是否为老顾客。
9. 系统打开到库存系统的连接。
10. 系统通过库存系统请求当前库存量。
11. 库存系统返回当前库存量。
12. 系统验证购买商品的数量是否小于库存量。

... etc.

上述描述中存在的问题：

◆对用例的描述过于冗长，可以采用更为简洁的描述方式，如合并类似的数据项（步骤3至步骤5），提供抽象的高层描述（步骤9至12）等。

◆改进的描述可以如下所示：

Use Case: Buy Something

参与者: user(或Customer)

主事件流：

1. 顾客使用ID和密码进入系统。
2. 系统验证顾客身份。
3. 顾客提供个人信息(包括姓名、地址、电话号码), 选择要购买的商品及数量。
4. 系统验证顾客是否为老顾客。
5. 系统使用库存系统验证要购买的商品数量是否少于库存量。

... etc.

7.4 Use Case图的建立步骤

(1) 找出系统外部的参与者和外部系统，确定系统的边界和范围；

(2) 确定每一个参与者所期望的系统行为；

(3) 把这些系统行为命名为Use Case；

(4) 使用泛化、包含、扩展等关系处理系统行为的公共或变更部分；

(5) 编制每一个Use Case的脚本；

(6) 绘制Use Case图；

(7) 区分主事件流和异常情况的事件流，可以把表示异常情况的事件流作为单独的Use Case处理；

(8) 细化Use Case图，解决Use Case间的重复与冲突问题。

7.5 例题

下面陈述对ATM系统的需求：

某银行拟开发一个自动取款机系统，它是一个由自动取款机、中央计算机、分行计算机及柜员终端组成的网络系统。ATM和中央计算机由总行投资购买。总行行分软件开发成本由各个分行分摊。

自动取款网有责营业发行计算下个分行分别设在柜各分行的多台ATM，分算机和的行分及供厅属摊。

银行可开户储款通户交相某储户或常提来应个柜以新一的现分户，交的的账员用账个存金行的使现户储款或计事端向可有务付，护终票也拥事或信维员支户能款，通且柜或储可取票机并用金。户或支算务处自以多输给分账理己从个进储行户。储拥自账柜户计户有己户员现算提的的。终金机交某账柜端。具的个户员，柜体储账中负接员处蓄户取责收终理事务。储内存款把储端针对

拥有银行账户的储户有权申请领取现金兑换卡。使用现金兑换卡可以通过ATM访问自己的账户。目前仅限于用现金兑换卡在ATM上提取现金（即取款），或查询有关自己账户的信息（例如，某个指定账户上的余额）。将来可能还要求使用ATM办理转账、存款等事务。

所谓现金兑换卡就是一张特制的磁卡，上面有分行代码和卡号。分行代码唯一标识总行下属的一个分行，卡号确定了这张卡可以访问哪些账户。通常，一张卡可以访问储户的若干个账户，但是不一定能访问这个储户的全部账户。每张现金兑换卡仅属于一个储户所有，但是，同一张卡可能有多个副本，因此，必须考虑同时在若干台ATM上使用同样的现金兑换卡的可能性。也就是说，系统应该能够处理并发的访问。

当用户把现金兑换卡插入ATM之后，ATM就与用户交互，以获取有关这次事务的信息，并与中央计算机交换关于事务的信息。首先，ATM要求用户输入密码，接下来ATM把从这张卡上读到的信息以及用户输入的密码传给中央计算机，请求中央计算机核对这些信息并处理这次事务。中央计算机根据卡上的分行代码确定这次事务与分行的对应关系，并且委托相应的分行计算机验证用户密码。如果用户输入的密码是正确的，ATM就要求用户选择事务类型（取款、查询等）。当用户选择取款时，ATM请求用户输入取款额。最后，ATM从现金出口吐出现金，并且打印出账单交给用户。

7.4.1 确定系统边界

- 确定系统的边界，就是了解系统的应用范围，把功能确定下来。
- 要确定系统包含哪些功能，忽略哪些功能。

64

- 确定与系统直接交互的外部对象。
- 参与者可以是人、外部设备和其他软件系统。

65

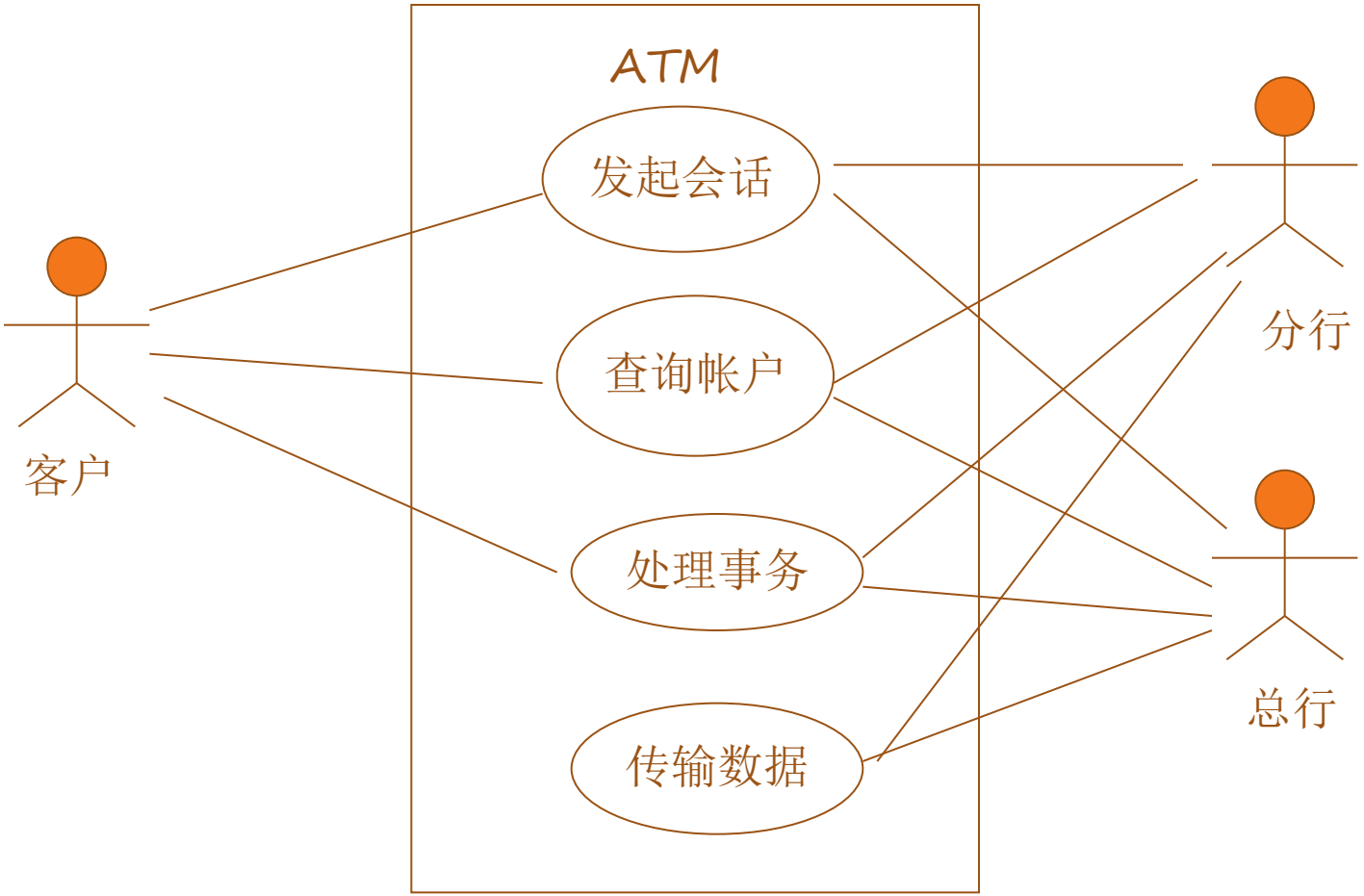
7.4.3 寻找用例

- 用例将系统功能划分成少数离散单元，所有的系统行为都必须处在某种用例之下。

ATM系统用例：

- 发起会话。ATM建立用户标识，并提供一列帐户和活动；
- 查询帐户。系统为帐户提供通用数据，例如当前余额、最后一笔交易的日期以及上一张报表的邮寄日期；
- 处理事务。ATM系统执行会影响帐户余额的活动，如存款、提款和转帐。ATM确保所有完成的交易最终都要写进银行数据库。
- 传输数据。ATM使用总行的设备来与相应的分行计算机通信。

66



67

7.4.4 寻找初始和终止事件

确定每个用例的初始事件以及其包含多少个终止事件。

- 用例将系统的功能拆分成离散的单元，并显示了每一个单元所涉及到的参与者，但他们并没有清晰地显示行为。要理解行为，就必须了解每一用例的执行顺序。
- 寻找：发起每个用例的开始事件；确定哪个参与者发起事件；定义它发送给系统的事件；确定一个或者多个终止事件。

68

ATM例子：

(1) 发起会话。初始事件是客户插入现金卡。有两个终止事件：系统留存现金卡或者系统返回现金卡。

(2) 查询帐户。初始事件是客户请求帐户数据。终止事件是系统给客户递交帐户数据。

(3) 处理事务。初始事件是客户发起事务。有两个终止条件：终止事务或者提交事务。

(4) 传输数据。初始事件是客户请求数据；另一种可能的初始事件是从网络、电源或者另一种故障中恢复出来。终止事件是数据成功传送。

69

7.4.5 准备场景

场景是在在一组交互对象之间发生的一系列事件。

◆当系统中对象和外部代理(例如用户、传感器或其它任务)之间有信息交换的时候，就会发生事件。被交换的信息就是事件的参数。

◆ATM场景例子：

70

发起会话

ATM请求用户插卡。
用户插入现金卡。
ATM接受现金卡，读取它的序列号。
ATM请求输入密码。
用户输入”1234”。
ATM通过总行和分行联系来效验密码。
ATM显示帐户和菜单命令。
....
用户选择命令终止会话。
ATM打印收据，返还现金卡，并请求用户拿走它们。
用户拿走现金和收据。
ATM请求用户插入新金卡

查询帐户

ATM显示帐单和命令菜单。
用户选择查询帐户。
ATM与返回数据的总行与分行联系。
ATM给用户显示帐户数据。
ATM显示帐户和命令菜单。

71

处理事务

ATM显示帐户和命令菜单。
用户选择帐户提款。
。 。 。 。

传输数据

ATM从总行请求数据。
总行接受请求，并将其转发给相应的分行。
分行接受请求，并检索要求的数据。
分行将数据发送到总行。
总行将数据路由给ATM。

72

7.5 练习1

阅读下面与网络数据备份相关的功能描述，选择适当的语句填入用例图的空格(a)~(g)中（可以选择多次）

将网络数据备份软件所需的功能用归纳为用例图。

这个网络数据备份软件有两种下载的方法。一种是随时开始备份当前磁盘的数据，另一种是先设定好时间和文件路径进行预约备份。

预约备份时，有两种方法。一种为手动设置日期和文件路径；另一种为通过其它应用系统从需要备份数据的配置系统中获取文件路径信息，然后从一览信息中选择要预约备份的文件路径。

到了路径下文件开始更新的时间，就开始自动备份已经预约好的文件。

可以从一览显示画面确认已经完成完毕的路径，并可从该画面选择要重放或删除的备份数据。

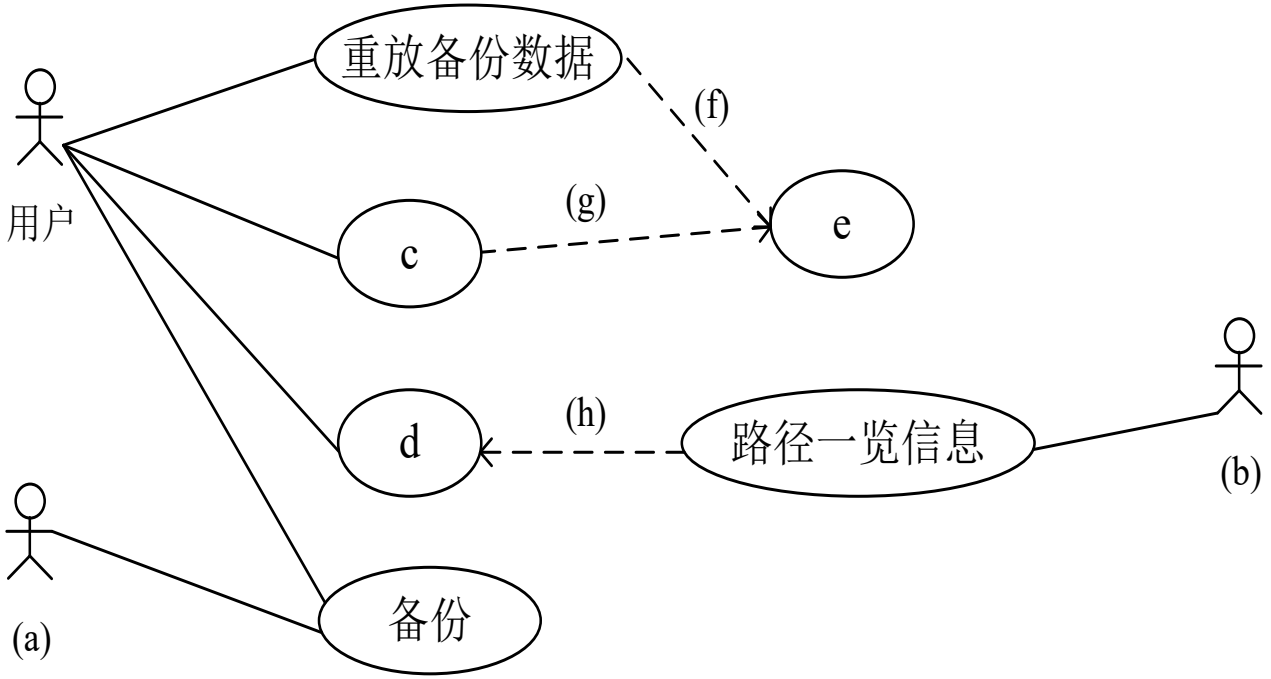
73

选项：(1)文件路径信息　(2)需要备份数据的配置系统　(3)定时器　(4)存储设备

(5)确认备份的文件路径一览　(6)选择要备份的文件路径　(7)设置时间

(8)设置文件路径　(9)删除备份的文件路径　(10)预约备份

(11)extend　(12)include



74

7.5 练习1答案

◆答案：a-(3) b-(2) c-(9) d-(10) e-(5) f-(12) g-(12) h-(11)

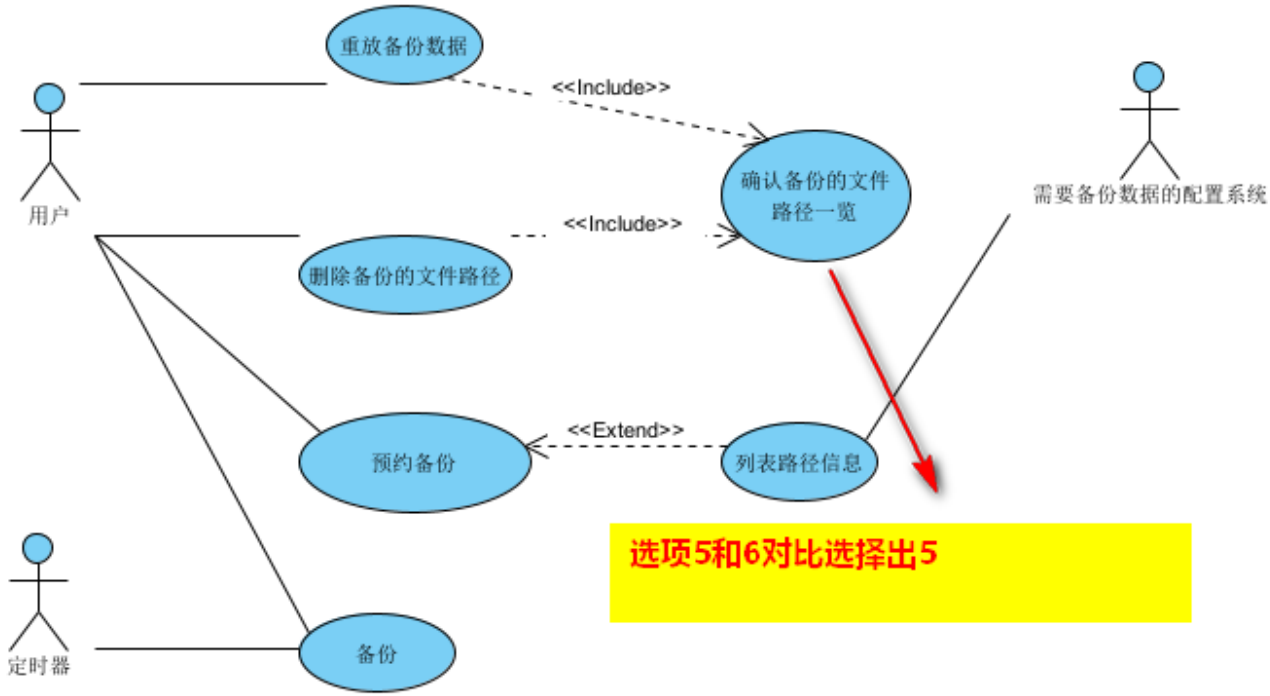
◆参与者：

- 手动备份的参与者应该是用户。
- 预约备份需要“时间参与者”。
- 需要从“其他应用系统”中取得数据，“其他应用系统”或者“需要备份数据的配置系统”

◆用例：

- 备份
- 预约备份
- 获取文件路径信息
- 确认文件路径
- 删除备份
- 重放备份

◆用例：动词开头



7.5 练习2

考虑一个实际的书店，例如新华书店类似，完成以下问题。

（a）列出结账系统中会用到的三类参与者。解释每一种参与者的相关性；

（b）一个用例是购买商品。从客户角度，列举出在类似的抽象层次上的另一个用例，并用一句话来总结每个用例的意图。

（c）给书店的结算系统绘制出一张用例图。

（d）给每个用例编制一个普通的场景。场景只是例子，不需要实现全部功能

（e）给每个用例绘制一个异常场景。

（f）对应用于(d)中的每个场景，绘制一个顺序图

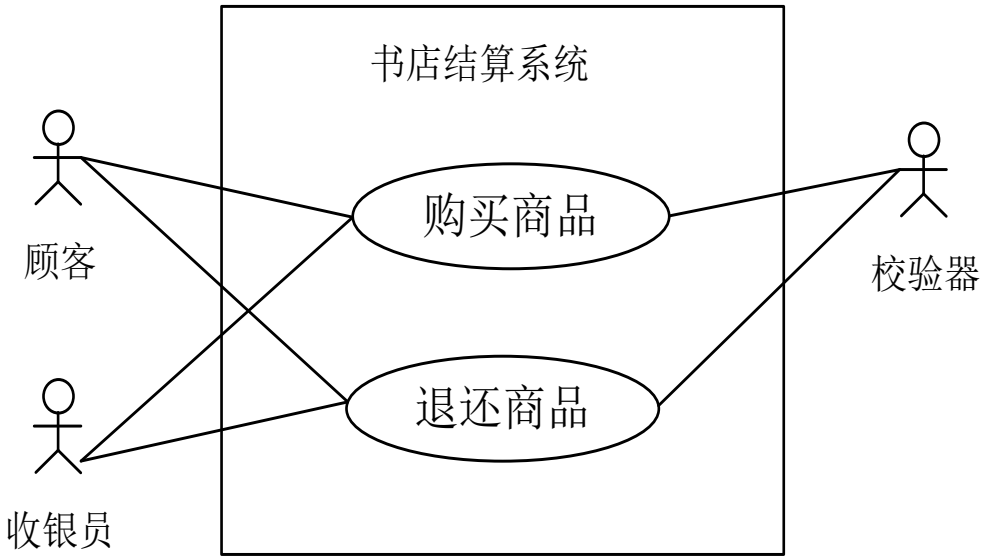
76

7.5 练习2-答案

(a)参与者包括：客户，购买商品的人。出纳员，被授权在收银台结算购买商品的一名雇员。支付校验器，批准使用信用卡或者借记卡的远端机器。

(b) 用例包括：购买商品，客户带着一项或者多项商品到收银台，为商品付费。退还商品，客户退还以前购买的商品，拿到退款。

(c)用例图



77

7.5 用例分析举例1：家庭安全系统

问题陈述：

系统由房主在安装时配置，通过系统控制窗口和键盘功能与房主交互进行。系统可以监控所有连接的传感器。

在安装过程中，每个传感器可以设置编号和类型，系统的启动和关闭必须有主人密码控制；传感器事件发生时，软件激活系统上的警报器，在设定的时间延时后，软件自动拨打设定的电话号码，并提供位置信息和事件性质，电话号码将每隔20秒重拨一次，直至电话接通。

系统运行时，系统控制窗口显示提示信息和系统状态。通过键盘可以控制系统运行。

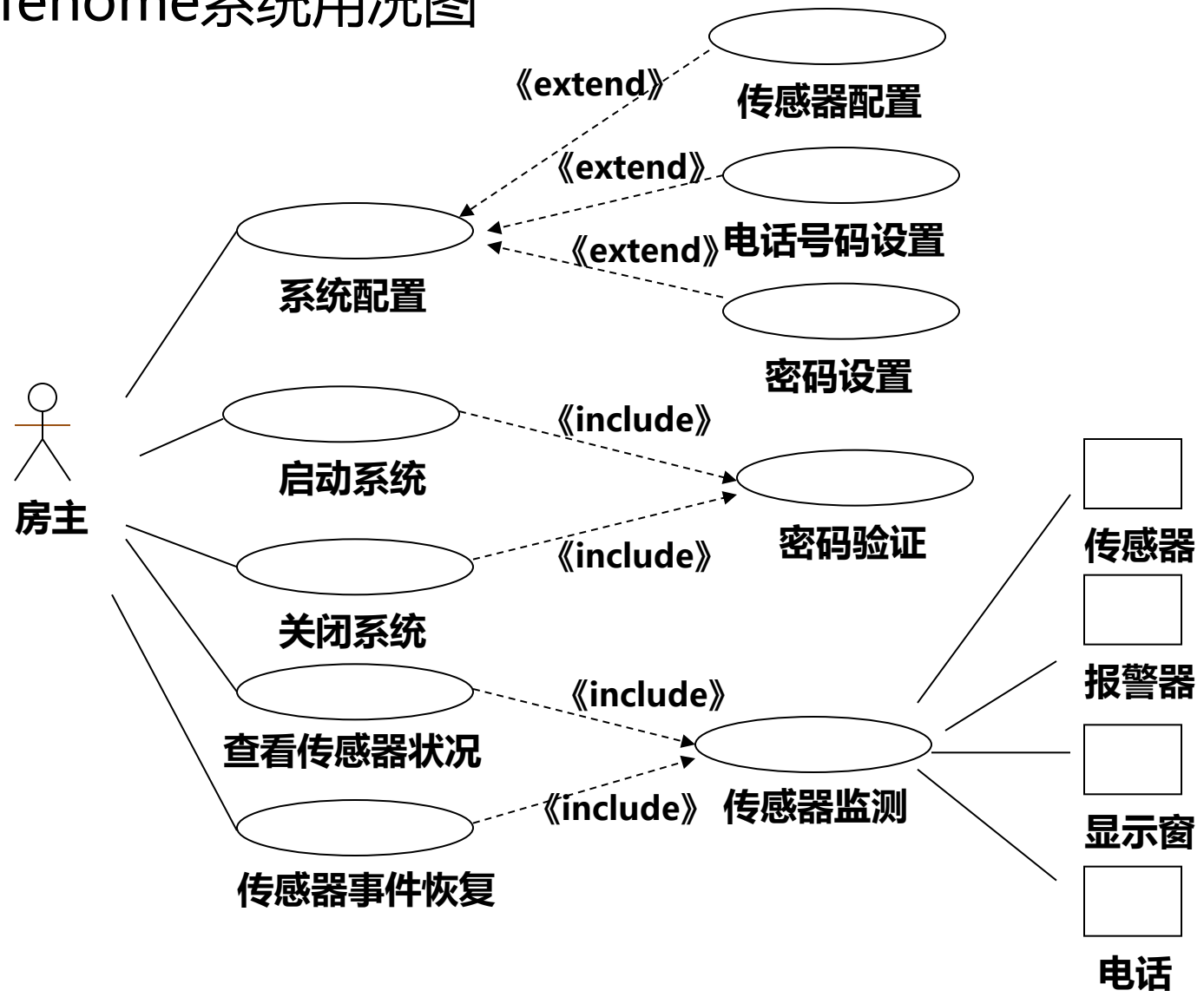
对问题陈述进行分析，从系统的**参与者**入手，分析**参与者**与**系统交互的功能。**

系统外部的参与者，包括人员、设备和其他系统

在此问题中包括：

房主	启动和使用系统的主动参与者
传感器	系统监控的主要参与者
警报器	系统操纵的参与者
电话	同上
控制窗口	虽然是标准设备，但有特殊定义

Safehome系统用况图



7.5 用例分析举例2：道路坑洼修复系统

西北工業大學 NORTHWESTERN POLYTECHNICAL UNIVERSITY

问题陈述：

市民可以登录网站报告坑洼的位置和严重程度。每个被报告的坑洼，将登记到“市政部门的修复系统”中，并被赋予一个标识号，按街道地址、大小（1-10）、位置（路中或路边等）、城区（地址所在城区）和修复理由的优先级（坑洼严重程度）存储起来。

市政人员确认坑洼情况属实后，填写派工单并派工修复坑洼。工单内容包括：位置和大小、修理队的人数、使用的修理装备、修复所用的时间、坑洼状况（正在工作、已被修理、临时修理、未修理）、使用填料的数量和修理的开销（由使用的时间、人数、使用的材料的装备计算得到）。

坑洼经修复后，可输出坑洼的损害报告，包括市民的姓名、地址、电话号码、损害类型和修复总费用。

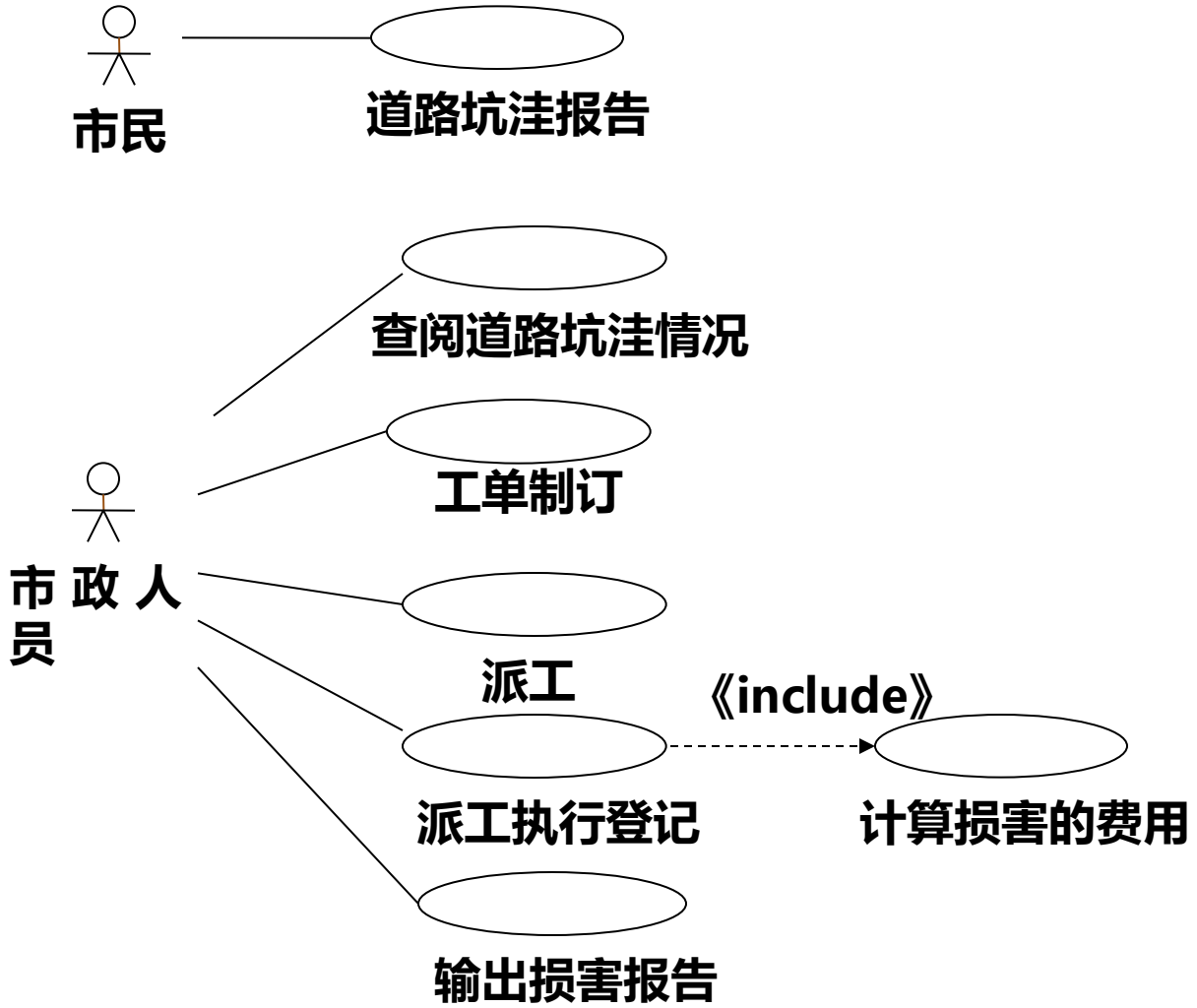
对问题陈述进行分析，从系统的参与者入手，分析参与者与系统交互的功能。

在此问题中，系统外部的参与者包括：

市民---------- 登记报告坑洼

市政工作人员------填写工单，派工，并汇总和输出修复情况及费用。

道路坑洼修复系统用例图：



7.5 练习：电话客户信息系统

问题陈述：

是电话公司开发管理电话客户信息的交互式网络系统，功能如下：

1. 浏览电话客户信息：任何使用Internet的网络用户，都可以浏览电话客户信息（姓名、住址、电话号码等）。

2. 登录：电话公司授予每个电话客户一个帐号。授权了的电话客户，可以使用系统提供的页面来设置个人密码，并使用帐号和密码向系统注册。

3. 修改个人信息：电话客户向系统注册后，可以发送电子邮件或者使用系统提供的页面，对个人信息进行修改。

4. 删除电话客户信息：只有公司管理人员才能删除不再接受公司服务的电话客户信息。

对问题陈述进行分析，从系统的参与者入手，分析参与者与系统交互的功能。

在此问题中，系统外部的参与者包括：

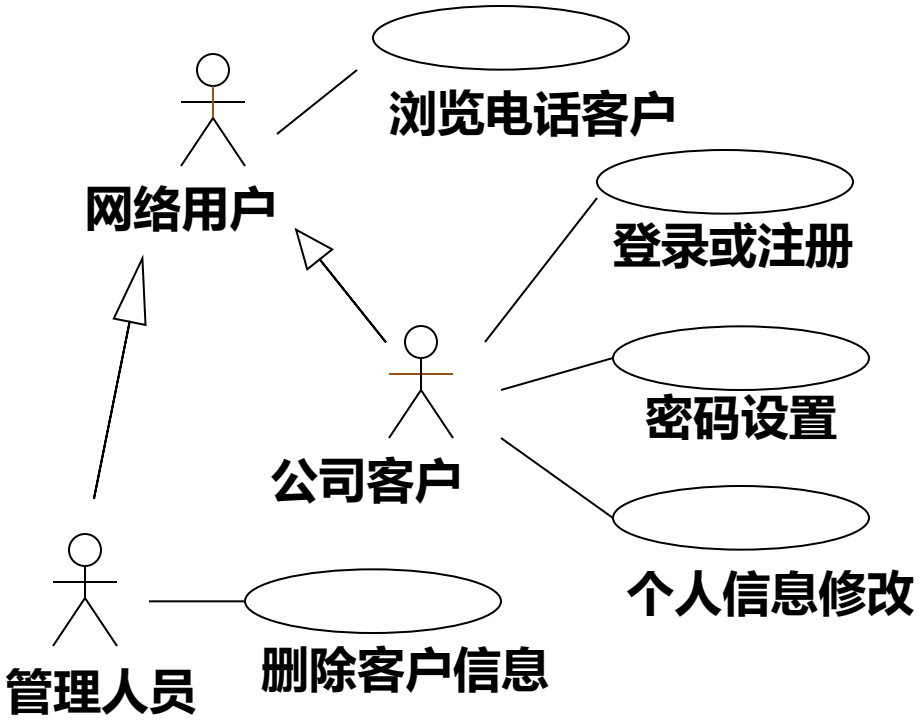
所有使用Internet的网络用户------- 查询电话客户信息

电话客户-------用指定帐户密码向系统注册，并可修改个

人信息。

公司管理人员-------删除电话客户信息

电话客户信息系统用例图：



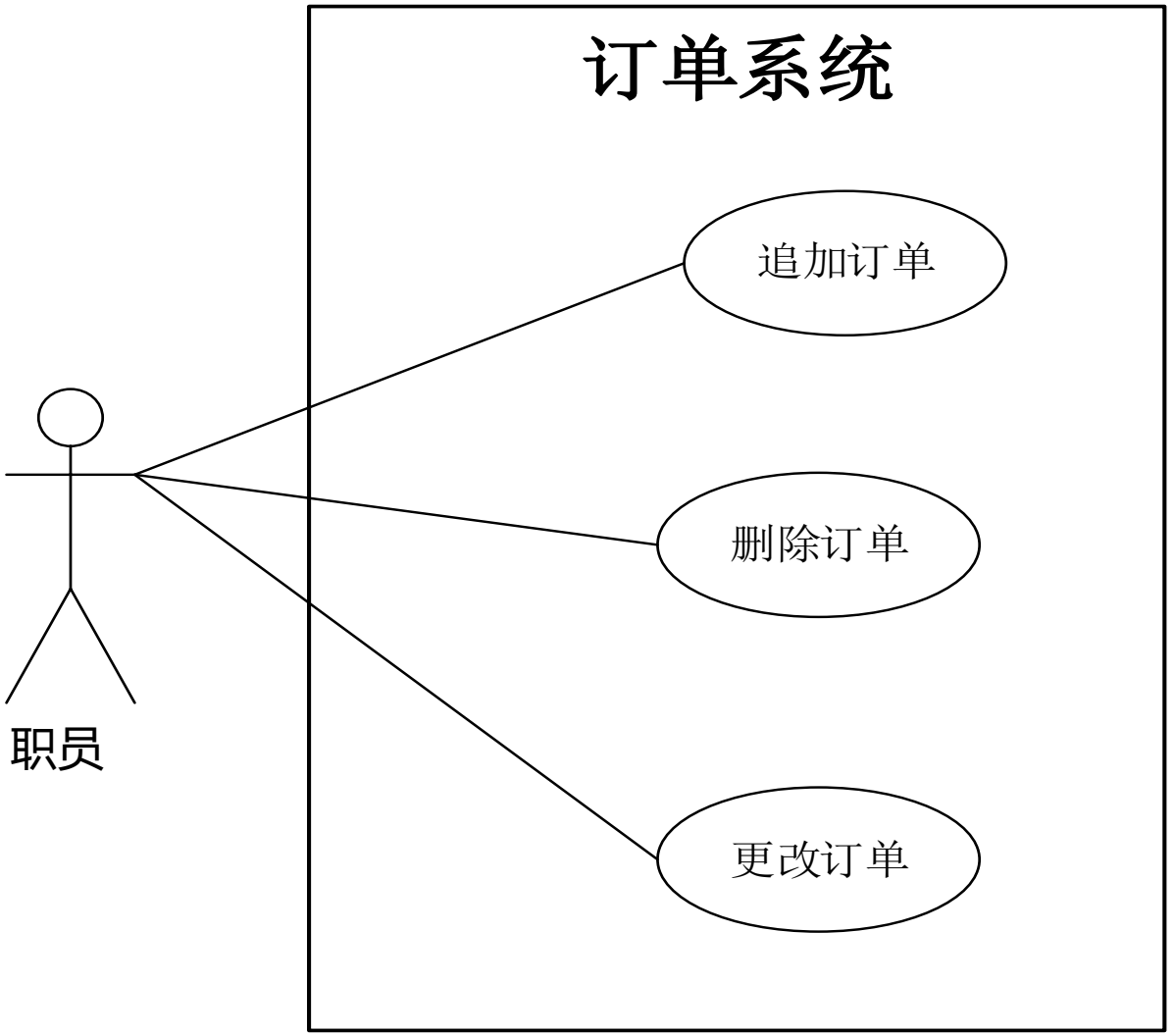
练习题

- 试画用例图描述以下内容：

  某销售公司准备开发一个订单管理系统。
  该公司的职员可以利用该订单系统进行订单的追加，删除和修改。

  请试着用用例图来描述该系统。

练习题参考答案

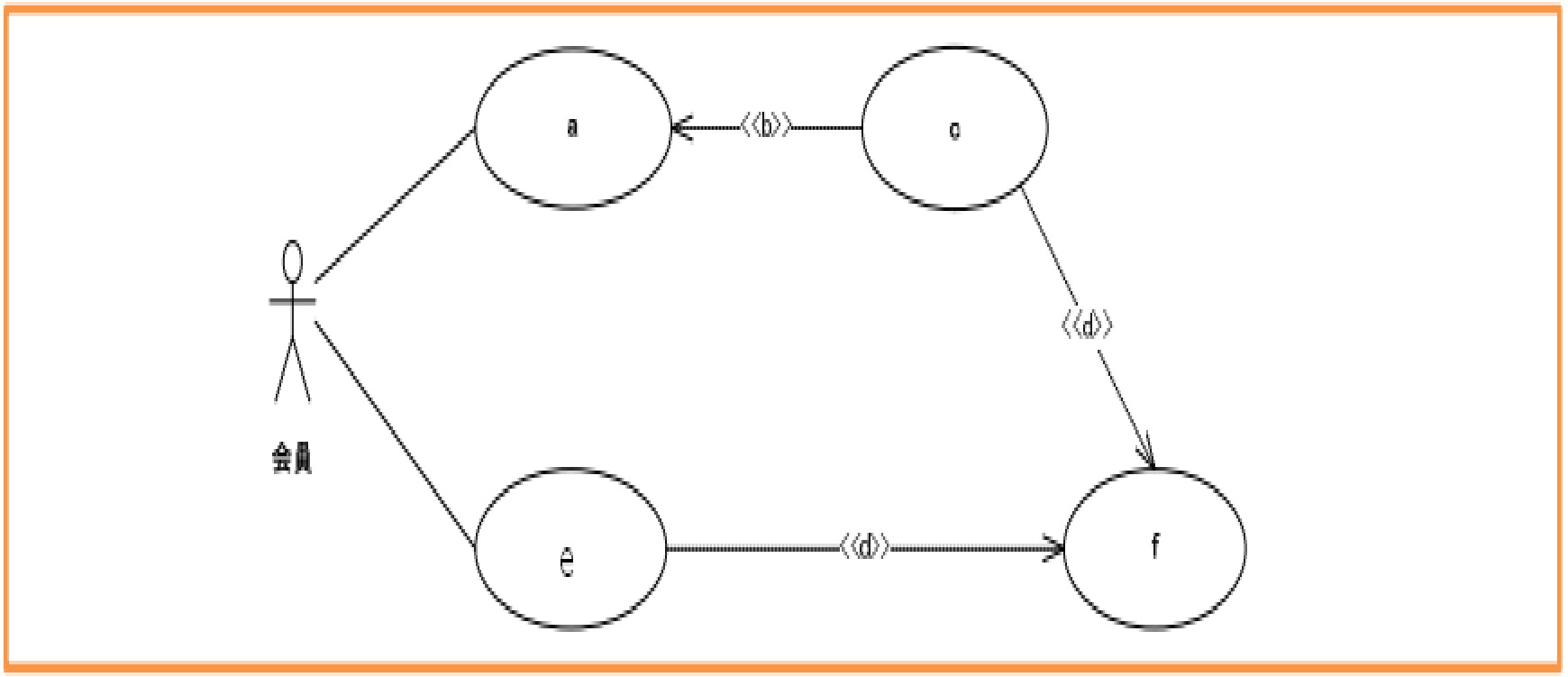


用例练习(UMTP L2 Sample)

- 考虑一个某剧场的售票系统。当前已经提出如下要求：
  - 如果有空位，会员可以预订门票。
  - 已预订车票的会员可在购票时指定座位。指定座位时可以从空座中选择喜欢的座位。
  - 今后可能会提供在订票时指定座位的服务。
- 选择用例图中满足当前需求的（a）至（f）项，完成模型。

用例练习(UMTP L2 Sample)（续）

- (1) create （2) extend （3) include （4) use
- (5) 会员登录 （6) 有无空座位? （7) 预订门票 （8) 门票预订完成
- (9) 购票 (10) 选择喜欢的座位 (11) 查询空座位 (12) 指定座位



用例练习(UMTP L2 Sample)（续）

◆ 解答 = a-(9) b-(2) c-(12) d-(3) e-(7) f-(11)

(1) create (2) extend (3) include (4) use

(5) 会员登录 (6) 有无空座位? (7) 预订门票 (8) 门票预订完成

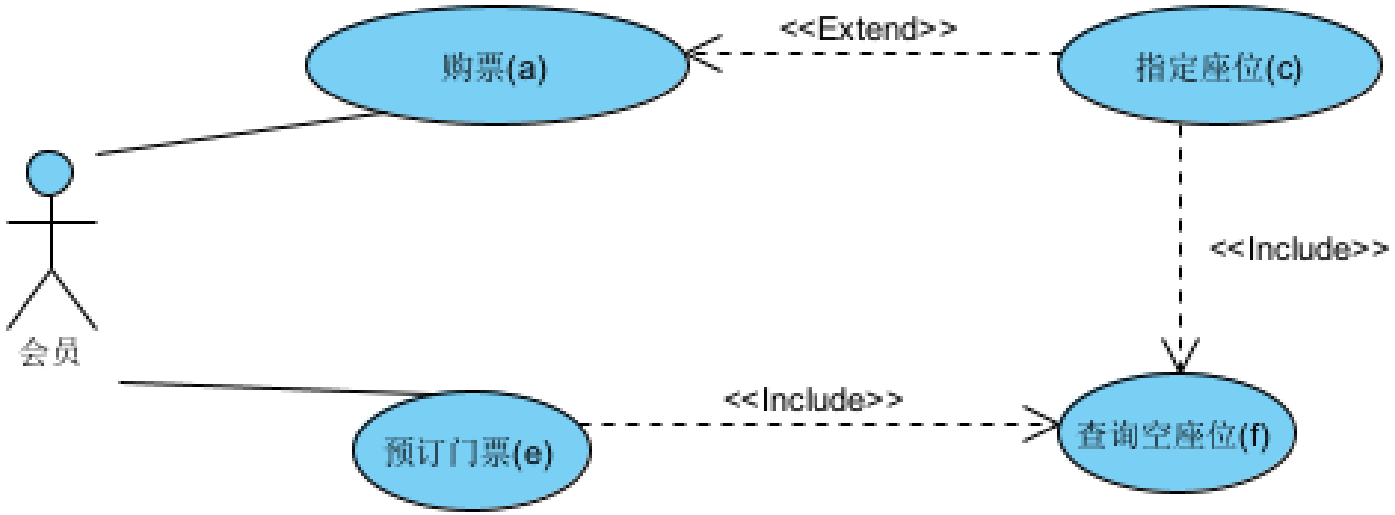
(9) 购票 (10) 选择喜欢的座位 (11) 查询空座位 (12) 指定座位

◆参与者:

- ■会员

◆用例

- ■预订
- ■购票
- ■选择座位
- ■查询空位



◆预订时指定座位是将来功能，因此图中没有体现。

总结

- 用例图

- 用例图是在系统开发的初期阶段，描述站在用户的角度上看系统提供给用户的可使用的功能。

- 参与者是与系统进行交互的角色，不是某个具体的个人。可以是人，硬件，或者外部已经存在的某个系统。共同点是要与系统有直接的信息交互。

- 用例图和用例描述合称为用例模型。

- **用例规范（Use Case Specification）**
- 这是一份文档，详细描述了每一个用例（功能或业务流程）。用例规范通常包括：
  - **用例名称**：简洁描述功能的名称。
  - **参与者**：与用例交互的用户或系统。
  - **触发条件**：描述什么条件下用例会启动。
  - **前置条件**：用例开始前必须满足的条件。
  - **后置条件**：用例执行完后，系统应处于的状态。
  - **主成功场景**：描述用例的正常执行流程。
  - **扩展或备选流程**：如果有异常或特殊情况发生时的流程。
  - **特例或异常流程**：当错误发生时应如何处理。
- 用例规范提供了用例的全面视图，不仅包括正常的业务流程，也包括可能的异常处理和特殊情况。
- **用例脚本（Use Case Script）**
- 这通常是一个更加详细的步骤说明，类似于一个剧本，包含用户与系统之间具体的交互。
  - 它会为每个步骤编号，并详细描述用户执行的动作和系统的响应（例如，弹出消息、页面导航等）。
  - 用例脚本通常是用例测试的基础，可以用以创建测试脚本。
- **用例场景（Use Case Scenario）**
- 这是一个特定的故事，描述了一个用例的单一执行路径。
  - 它通常基于用例脚本，提供了一个实例化的视图，就好比从用例规范中抽出一个具体的故事 用例规范（Use Case Specification）、用例脚本（Use Case Script）、用例场景（Use Case Scenario）和事件流（Event Flow）确实容易让人混淆，因为它们都是用于描述软件系统如何响应外部交互的方法。我们可以通过它们在用例文档中的作用来区分它们。
- 用例规范（Use Case Specification）：
  - 这是对整个用例的详细描述。通常包括用例的名称、参与者、前提条件、后置条件、主成功场景（或称基本流程）、备选场景（或称分支流程）、异常处理和任何特别需求。它是用例相关所有信息的集合点。
- 用例脚本（Use Case Script）：
  - 通常是用例规范的一部分。它是详细描述特定用例如何一步步执行的脚本。用例脚本按时间顺序从用户开始交互到操作完成描述了所有必需的详细步骤。
- 用例场景（Use Case Scenario）：

◆ 用例规范、用例脚本、事件流和用例场景是软件工程中需求分析和测试设计阶段常用的几个概念，它们之间存在密切的关联，共同用于确保软件产品的功能符合用户需求。下面是对这几个概念简要的描述以及它们之间的关系：

1. **用例规范（Use Case Specification）**： 用例规范是一个详细的文档，描述了系统如何与外部实体（如用户、其他系统）交互以完成特定的目标或任务。它从用户的角度出发，定义了系统的功能需求，包括前置条件、后置条件、基本流（正常流程）和备选流（异常流程或不同选择路径）。用例规范为开发团队提供了实现功能的具体指导，也为测试团队提供了测试的基础。

2. **用例脚本（Use Case Script）**： 用例脚本是从用例规范中提取的、更为具体的操作步骤序列，通常用于指导测试执行。它详细列出了执行某个用例时，用户或系统需要进行的每一步操作及其预期结果。用例脚本使得测试过程更加标准化和可重复，有助于确保测试的准确性和一致性。

3. **事件流（Event Flow）**： 事件流是描述在执行用例过程中，系统与用户或其他系统之间交互序列的一种方式。它包括了从开始到结束的所有可能的动作和响应，不仅涵盖基本流，还包括各种备选流。事件流帮助理解在不同情况下系统的行为，是构建用例脚本和进行测试设计的关键输入。

4. **用例场景（Use Case Scenario）**： 用例场景是对用例规范中描述的交互模式的一个具体实例化描述，它基于一个特定的上下文或目标，详细说明了用户如何使用系统来达成某个具体目标的过程。一个用例可以有多个场景，每个场景都是一种可能的事件流展现，涵盖了从简单到复杂的各种情况。场景帮助团队更好地理解和设计测试案例，确保软件在不同情境下都能正确工作。

◆ **关系总结**：

- **用例规范**是最上层的需求描述，为所有后续的详细设计提供框架。
- **事件流**是在用例规范基础上进一步细化的交互流程，展示系统行为的全貌。
- **用例脚本**基于事件流，是具体可执行的测试步骤，确保测试的实施与验证。
- **用例场景**是对用例规范的实际应用示例，通过不同的场景来覆盖用例的各种可能情况，帮助全面测试系统功能。

◆ 这些概念相互支持，共同构成了从需求分析到测试执行的完整链条，确保软件开发的质量和用户需求的满足。